

Machine Learning



Pr. Abdelalim SADIQ

Master Génie Logiciel pour le Cloud Computing

Mail: a.sadiq@uit.ac.ma

2024/2025

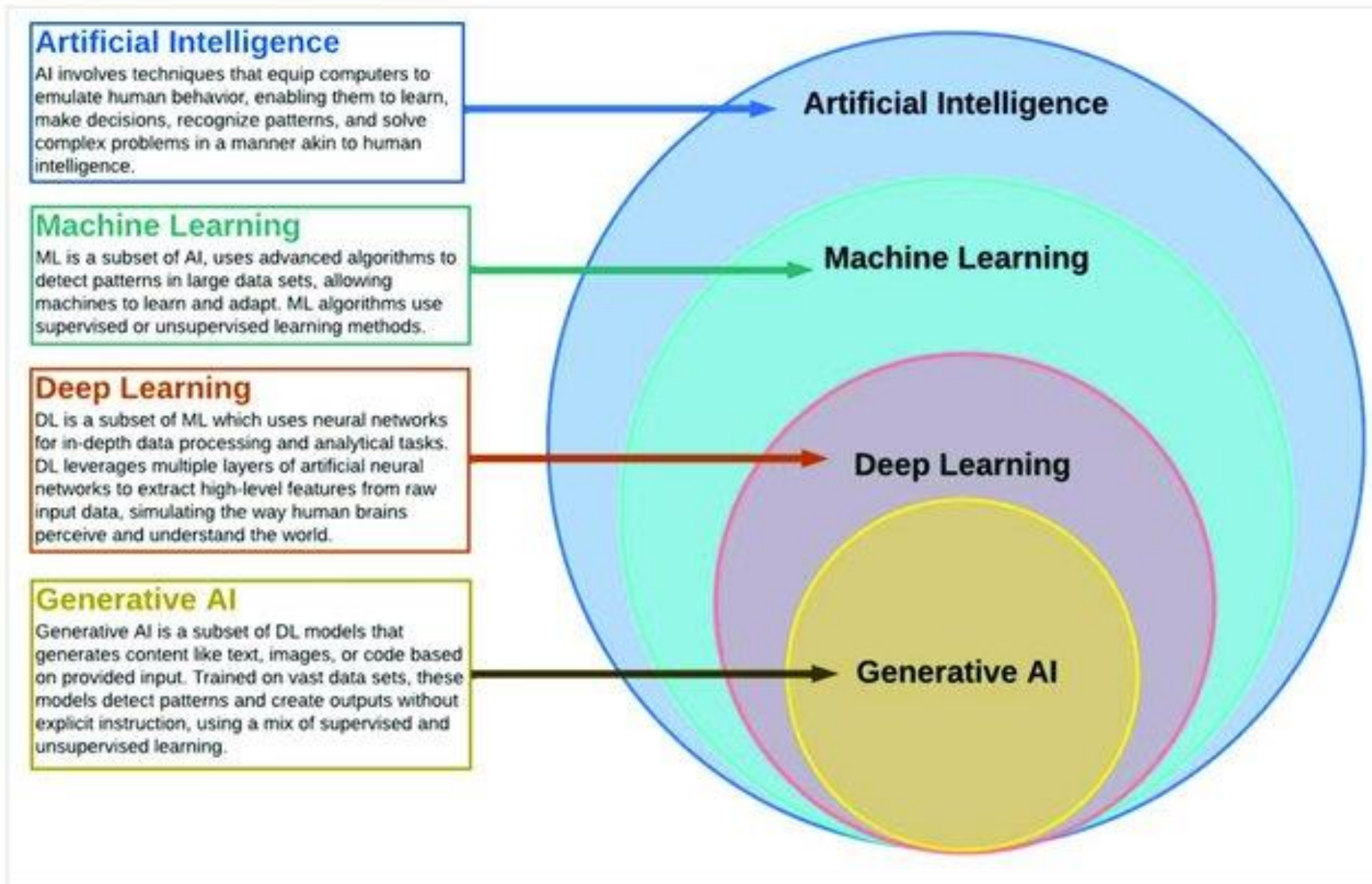
Plan de la présentation

- Introduction
- Rappel : Mathématiques pour ML
- Supervised Learning
 - Regression
 - Classification
- Unsupervised Learning
 - Clustering
 - Association
- Reinforcement learning

Mode d'évaluation

- La présence : **3 Absences = Module Non Validé**
- TP à rendre **chaque semaine sur Classroom**.
 - Implémentation des algorithmes **from scratch**.
 - Code utilisant les bibliothèques **TensorFlow, Keras**, etc.
- Projet sur une thématique spécifique.
- Examen final.

Introduction (1)



Introduction (2)

- **L'intelligence artificielle** est un domaine scientifique concerné par le développement d'algorithmes permettant aux ordinateurs d'apprendre sans être explicitement programmés.
- **Le machine learning** est un domaine de l'intelligence artificielle, qui consiste à programmer une machine pour que celle ci apprenne à réaliser des **tâches** en étudiant **des exemples** de ces dernières.
- Ces **Exemples** sont représentés par **des données** que la machine utilise pour développer un modèle, par exemple une fonction du type : $f(x) = ax + b$ (cas d'un modèle linéaire)
- Le machine learning consiste essentiellement à trouver cette fonction f :

- Speech Recognition

$$f(\text{audio waveform}) = \text{“كيف حالك”}$$

- Image Recognition

$$f(\text{cat image}) = \text{“Chat”}$$

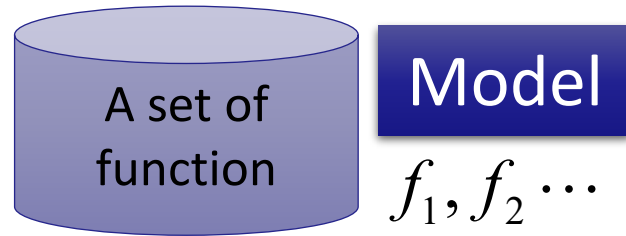
- Text Translation

$$f(\text{“How are you”}) = \text{“كيف حالك”}$$

Introduction (3)

- Image recognition

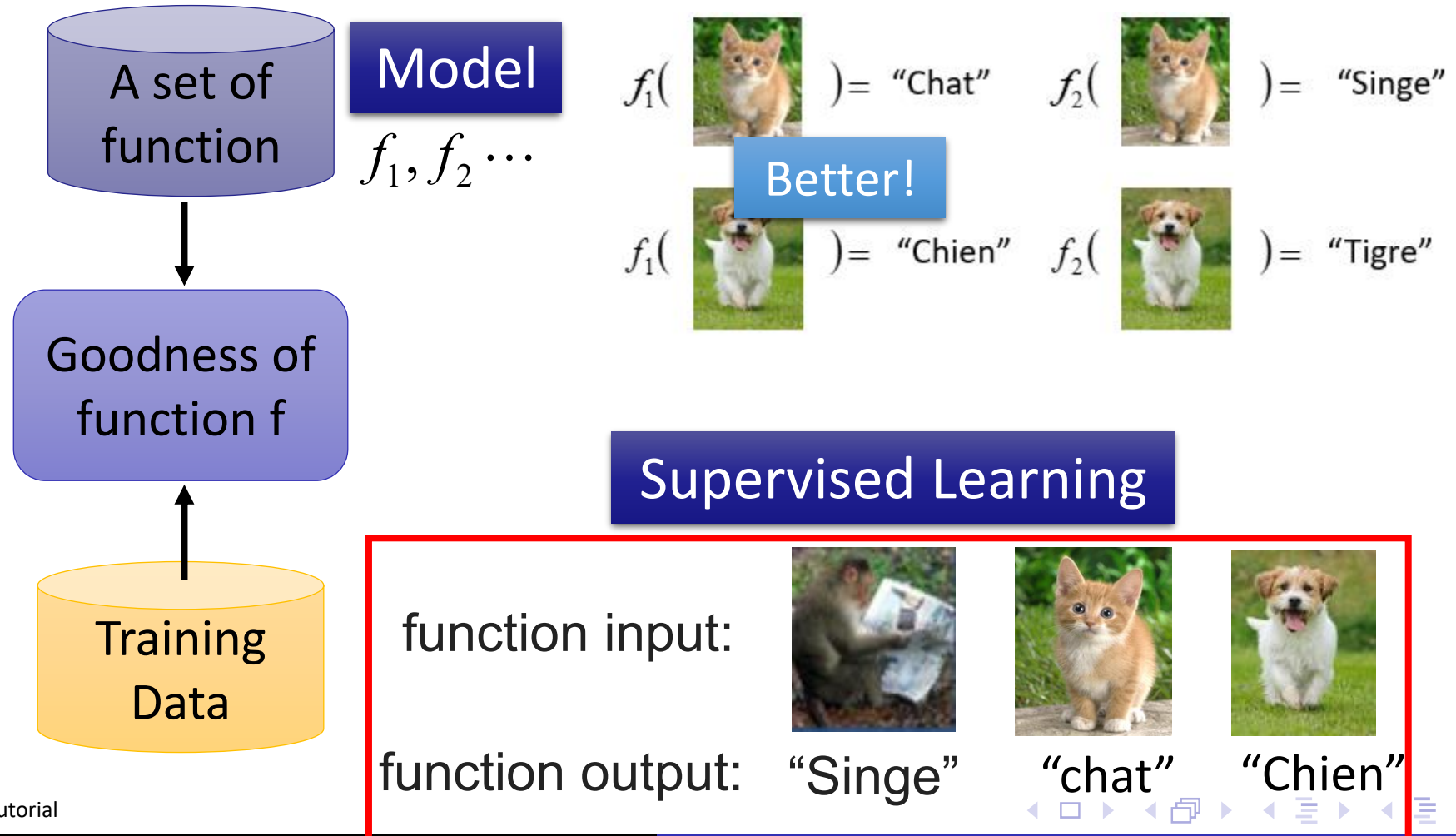
$$f\left(\text{img_cat}\right) = \text{"Chat"}$$



$$\begin{array}{ll} f_1\left(\text{img_cat}\right) = \text{"Chat"} & f_2\left(\text{img_cat}\right) = \text{"Singe"} \\ f_1\left(\text{img_dog}\right) = \text{"Chien"} & f_2\left(\text{img_dog}\right) = \text{"Tigre"} \end{array}$$

Introduction (4)

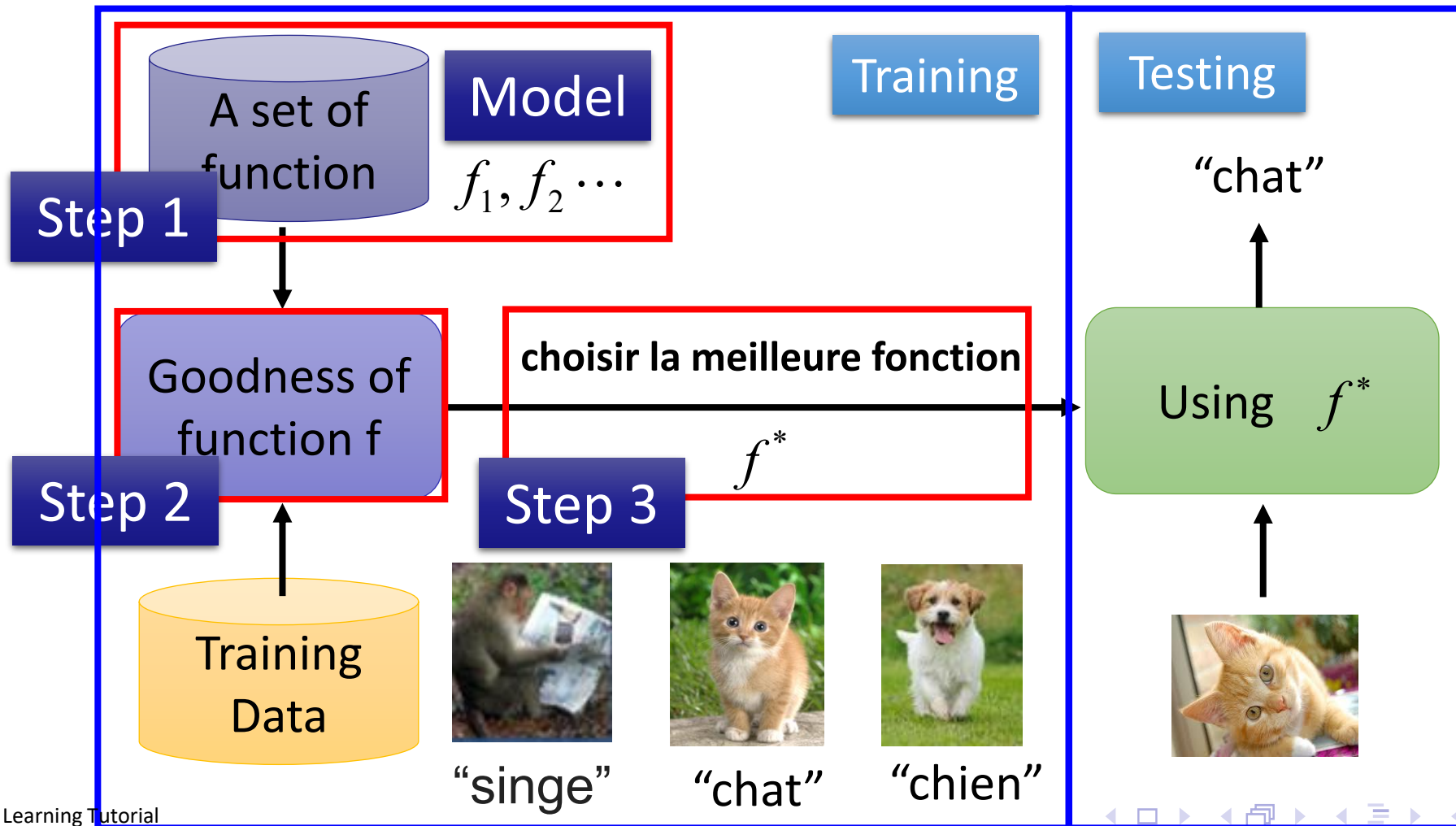
- Image recognition



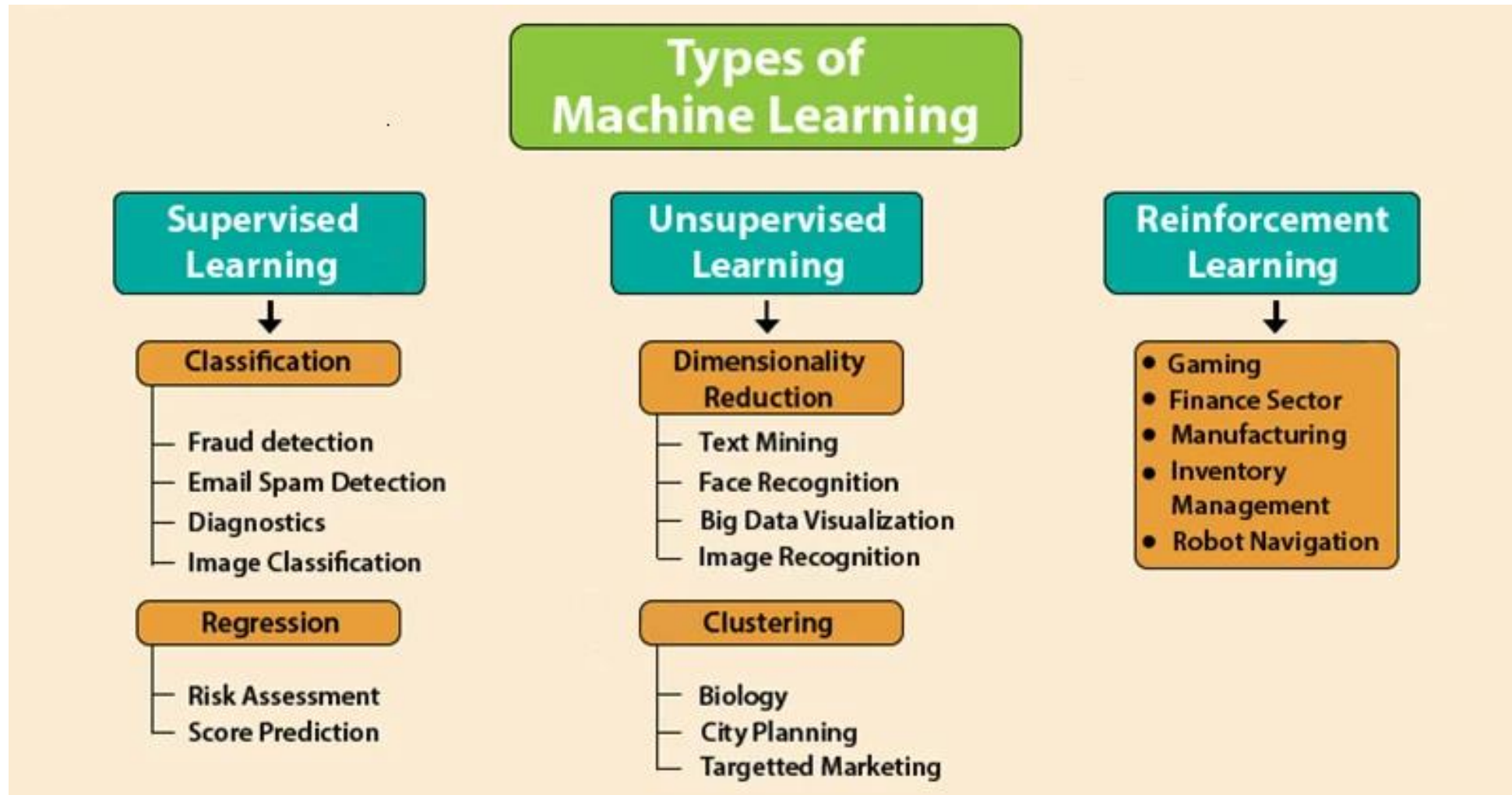
Machine Learning (4)

- Image recognition

$$f(\text{Image of a cat}) = \text{"Chat"}$$



Rappel : Machine Learning (1)



Rappel : Machine Learning (2)

- Machine learning supervisé peut se subdiviser en deux types :
 - **Classification** : la variable de sortie est une catégorie.
 - **Régression** : la variable de sortie est une valeur spécifique.
- Machine learning non supervisé
 - **Clustering** : l'objectif consiste à trouver des regroupements dans les données.
 - **Association** : l'objectif consiste à identifier les règles qui permettront de définir de grands groupes de données.
- Machine learning par renforcement
 - **Monte Carlo** : le programme reçoit ses récompenses à la fin de l'état « terminal ».
 - **Machine learning par différence temporelle (TD)** : les récompenses sont évaluées et accordées à chaque étape.

Apprentissage Supervisé (1)

- **L'apprentissage supervisé** consiste à fournir à une machine un grand nombre d'exemples pour qu'elle puisse les analyser et apprendre à partir d'eux. Pour maîtriser cette approche, il est essentiel de bien comprendre les quatre notions suivantes :
 - **Jeu de Données (Data Set)**
Ensemble d'exemples utilisés pour entraîner le modèle.
 - **Le Modèle et ses Paramètres**
Structure algorithmique qui apprend à partir des données en ajustant ses paramètres.
 - **Fonction de Coût**
Mesure de l'erreur entre les prédictions du modèle et les résultats attendus, utilisée pour optimiser les paramètres.
 - **Algorithme d'Apprentissage**
Procédure utilisée pour minimiser la fonction de coût en ajustant les paramètres du modèle.

Apprentissage Supervisé (2)

Jeu de Données (Dataset)

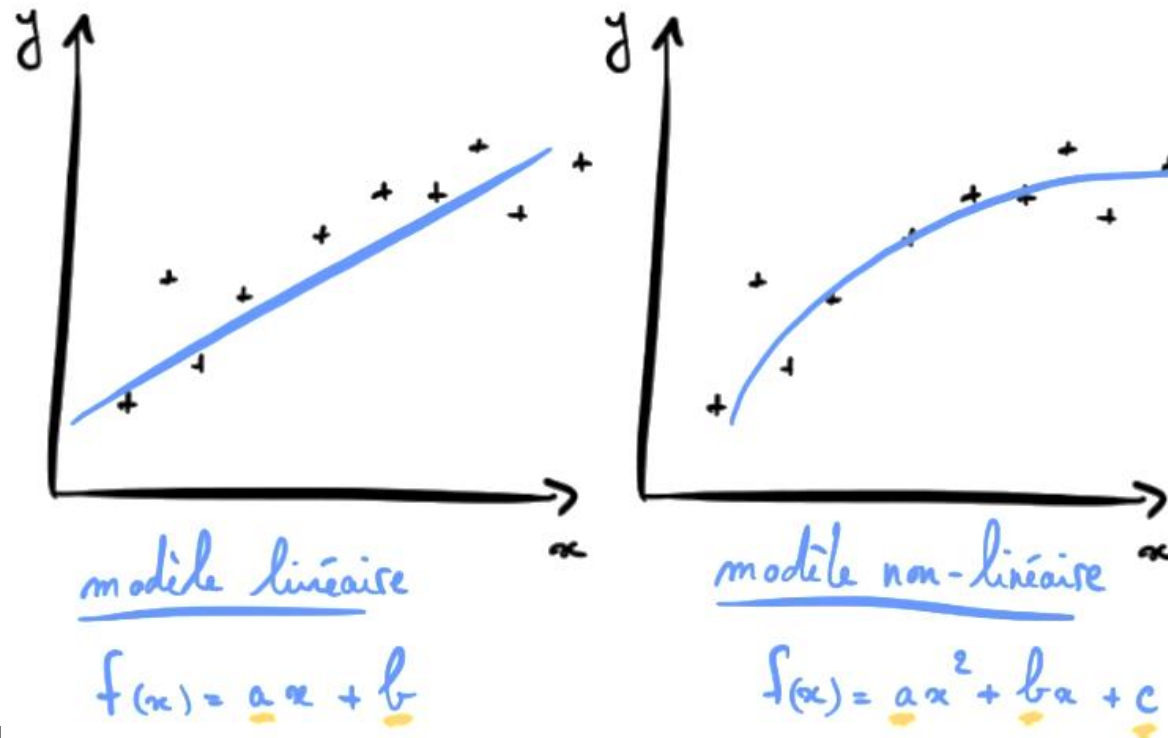
- En Machine Learning, les exemples (x,y) sont regroupés dans une structure appelé Dataset :
- **La variable y** est appelée *cible* (*target*). C'est la valeur que l'on cherche à prédire.
- **La variable x** est appelée *caractéristique* (*feature*). Un facteur influence la valeur de y . En général, un Dataset contient plusieurs caractéristiques $(x_1, x_2, \dots)$, qui sont regroupées dans une matrice **X**

Target y		Features x_1 x_2 x_3		
Prix		Surface m2	N chambres	Qualité
€	313,000.00	124	3	1.5
€	2,384,000.00	339	5	2.5
€	342,000.00	179	3	2
€	420,000.00	186	3	2.25
€	550,000.00	180	4	2.5
€	490,000.00	82	2	1
€	335,000.00	125	2	2

Apprentissage Supervisé (3)

Modèle

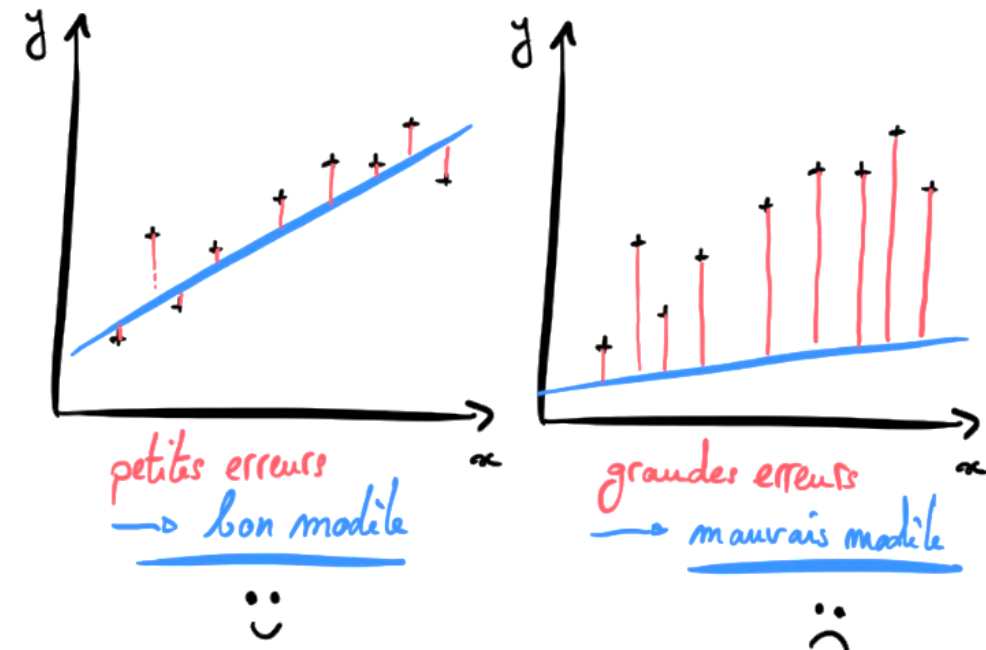
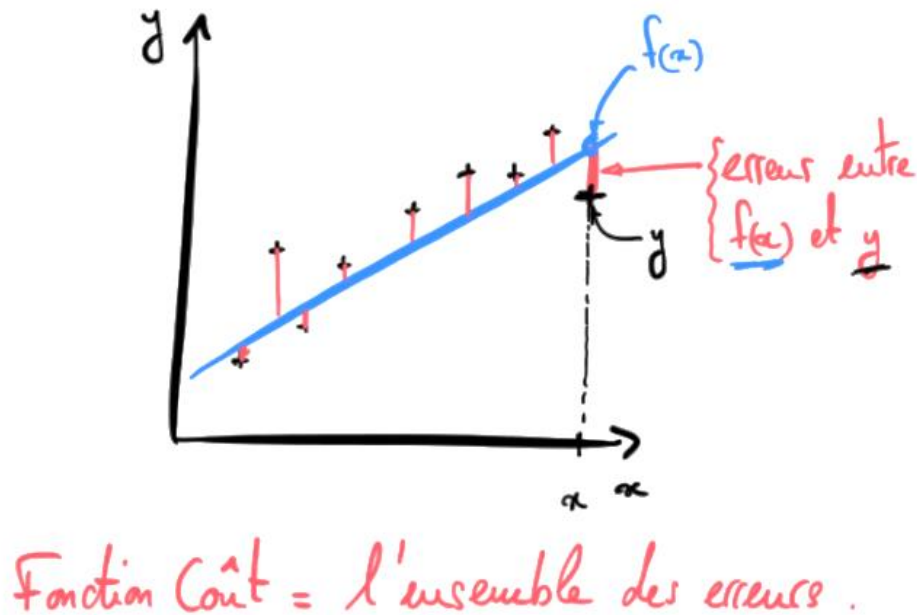
- En Machine Learning, un modèle est développé à partir du Dataset. Ce modèle peut être linéaire, comme illustré à gauche, ou non-linéaire, comme montré à droite. les critères pour choisir l'un ou l'autre de ces modèles dépendent de plusieurs facteurs, notamment la nature des données, la complexité des relations entre les variables, etc...



Apprentissage Supervisé (4)

Fonction coût

- Il est important de noter qu'un modèle génère des erreurs par rapport au Dataset. Ces erreurs sont regroupées sous ce que l'on appelle la **Fonction Coût**.
- Avoir un bon modèle, c'est avoir un modèle qui nous donne de petites erreurs, donc une **petite Fonction Coût**.



Apprentissage Supervisé (5)

Algorithme d'apprentissage

- Apprendre, c'est minimiser la Fonction Coût
- L'objectif principal en apprentissage supervisé est de trouver les **paramètres** du **modèle** qui minimisent la **Fonction Coût**.
- Pour ce faire, on utilise **un algorithme d'apprentissage** ou (**algorithme d'optimisation**), le plus courant étant l'algorithme de **Descente de Gradient**.

Modèle	Algorithme d'optimisation
Les modèles linéaires	la descente de gradient
Les arbres de décision	l'algorithme CART
Les SVM	la marge maximum

Mathématiques pour ML

Mathématiques pour ML

Algèbre Linéaire

- **Scalaire** est un simple nombre (une seule valeur). C'est une quantité qui ne dépend d'aucune direction, contrairement aux vecteurs ou matrices.

$$s = 5$$

- s est un scalaire qui pourrait représenter une température, une distance, ou n'importe quelle valeur numérique unique.
- **Vecteur** est une séquence ordonnée de nombres, qui peut être représentée comme une liste ou une colonne.

$$\mathbf{v} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

- Utilisation : Représentation d'un point dans l'espace n-dimensionnel

Mathématiques pour ML

Algèbre Linéaire

- **Matrice** : Une matrice est un tableau rectangulaire de nombres disposés en lignes et colonnes.

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

- Utilisation : Transformation linéaire, stockage des données en Machine Learning (features et observations).
- **Un Tenseur** est une structure de données généralisée qui peut représenter des scalaires (0-D), vecteurs (1-D), matrices (2-D), et des structures de dimensions supérieures.
- **Exemple de Tenseur 3-D :**

$$\mathbf{T} = \left(\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} \right)$$

- Ici, T est un tenseur de rang 3 (ou 3-D) composé de deux matrices 2x2.
- **Exemple dans Machine Learning :**
 - **Images** : Une image couleur est souvent représentée par un tenseur 3-D (largeur, hauteur, canaux de couleur).
 - Données d'une série temporelle ou d'une vidéo peuvent être représentées par un tenseur 4-D (nombre de frames, largeur, hauteur, canaux de couleur).

Mathématiques pour ML

Algèbre Linéaire

- **Addition de Vecteurs/Matrices** : Addition élément par élément entre deux vecteurs ou matrices de mêmes dimensions.

- Exemple :

$$\mathbf{u} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \mathbf{v} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$
$$\mathbf{u} + \mathbf{v} = \begin{pmatrix} 1 + 3 \\ 2 + 4 \end{pmatrix} = \begin{pmatrix} 4 \\ 6 \end{pmatrix}$$

- **Multiplication Matricielle** :

- Produit de deux matrices : Chaque élément de la ligne d'une matrice est multiplié par les éléments correspondants de la colonne de l'autre matrice, et les résultats sont additionnés.

- Exemple

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$
$$\mathbf{A} \times \mathbf{B} = \begin{pmatrix} (1 * 5 + 2 * 7) & (1 * 6 + 2 * 8) \\ (3 * 5 + 4 * 7) & (3 * 6 + 4 * 8) \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

Mathématiques pour ML

Algèbre Linéaire

- **Transpose d'une Matrice** : L'opération de transposition inverse les lignes et les colonnes d'une matrice.
- Exemple :

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad \mathbf{A}^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

- **Déterminant** est une valeur scalaire qui peut être calculée à partir d'une matrice carrée. Il donne des informations sur certaines propriétés de la matrice, comme si la matrice est inversible ou non.
- **Propriétés** :
 - Si le déterminant d'une matrice est zéro, la matrice est dite **singulière** (non inversible).
 - Le déterminant peut donner une idée du "volume" ou de la "zone" couverte par les vecteurs définis par la matrice.

Mathématiques pour ML

Algèbre Linéaire

- **Exemple de Calcul du Déterminant pour une Matrice 2x2 :** $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$

- Le déterminant de $\mathbf{A}$ est calculé comme : $\det(\mathbf{A}) = ad - bc$

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

- Exemple :

$$\det(\mathbf{A}) = (1 \times 4) - (2 \times 3) = 4 - 6 = -2$$

- **Exemple pour une Matrice 3x3 :**

$$\mathbf{B} = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{pmatrix}$$

- Le déterminant de $\mathbf{B}$ est calculé comme
$$\begin{aligned} \det(\mathbf{B}) &= 1 \times (4 \times 6 - 5 \times 0) - 2 \times (0 \times 6 - 5 \times 1) + 3 \times (0 \times 0 - 4 \times 1) \\ &= 1 \times 24 - 2 \times (-5) + 3 \times (-4) = 24 + 10 - 12 = 22 \end{aligned}$$

Mathématiques pour ML

Algèbre Linéaire

- **Produit scalaire** de deux vecteurs $\mathbf{x}$ et $\mathbf{y}$:

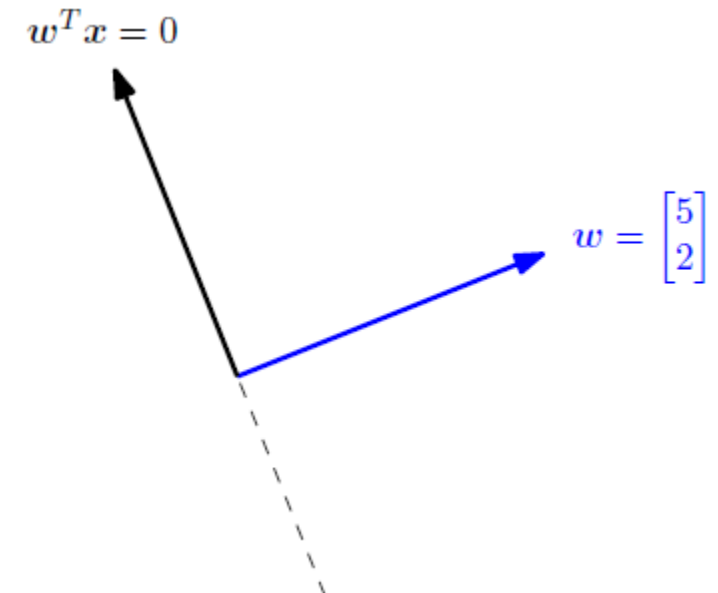
$$\mathbf{x}^T \mathbf{y} = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n$$

- La norme Euclidienne (L_2) d'un vecteur $\mathbf{x}$:

$$\|\mathbf{x}\| = \sqrt{\mathbf{x}^T \mathbf{x}}$$

- Deux vecteurs $\mathbf{x}$ et $\mathbf{y}$ sont orthogonaux si

$$\mathbf{x}^T \mathbf{y} = 0$$



Mathématiques pour ML

Probabilités et Statistiques

- **La probabilité** d'un événement est une mesure de la chance que cet événement se produise.

$$P(A) = \frac{\text{Nombre de cas favorables}}{\text{Nombre de cas possibles}}$$

- **Exemple** : Tirer un as d'un jeu de 52 cartes $P(\text{As}) = \frac{4}{52} = \frac{1}{13}$
- **Théorème de Bayes** : Permet de calculer la probabilité d'un événement en fonction des connaissances antérieures.

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

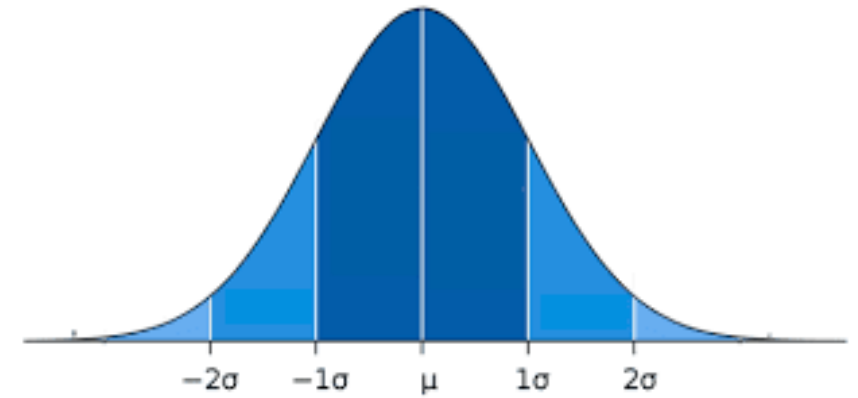
- **Exemple** : Calculer la probabilité qu'une personne ait une maladie donnée, sachant un test positif.

Mathématiques pour ML

Probabilités et Statistiques

- **Distribution Normale :**

- En forme de cloche, symétrique autour de la moyenne.
- Caractérisée par la moyenne (μ) et l'écart-type (σ).



- **Formule de la densité de probabilité :**

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- **Utilisation :** Modélisation des phénomènes naturels comme la taille des individus, les erreurs de mesure.

Mathématiques pour ML

Probabilités et Statistiques

- **Distribution de Bernoulli :**
- Modélise un événement binaire (succès ou échec).
- Paramètre p : Probabilité de succès.

- **Formule :**

$$P(X = x) = p^x (1 - p)^{1-x} \quad \text{où } x \in \{0, 1\}$$

- **Exemple :** Lancer une pièce (pile ou face).

Mathématiques pour ML

Probabilités et Statistiques

- **Moyenne (Mean)** : Moyenne arithmétique des valeurs.

$$\text{Moyenne} = \frac{1}{n} \sum_{i=1}^n x_i$$

- **Médiane** : Valeur qui sépare la moitié supérieure et inférieure des données.
 - **Exemple** : Dans [1,3,3,6,7,8,9][1, 3, 3, 6, 7, 8, 9][1,3,3,6,7,8,9], la médiane est 6.
- **Variance** : Mesure de la dispersion des données par rapport à la moyenne.

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

Mathématiques pour ML

Probabilités et Statistiques

- **Écart-Type (Standard Deviation)** : Racine carrée de la variance, donne une mesure de la dispersion en unités de données.

$$\sigma = \sqrt{\sigma^2}$$

- **Étendue (Range)** : Différence entre la valeur maximale et minimale.

$$\text{Étendue} = \text{Max} - \text{Min}$$

- **Écart Interquartile (Interquartile Range - IQR)** : Différence entre le 3ème quartile (Q3) et le 1er quartile (Q1).

$$\text{IQR} = Q3 - Q1$$

Mathématiques pour ML

Calcul Différentiel et Intégral

- **Dérivées** : La dérivée d'une fonction mesure la manière dont la fonction change lorsque sa variable d'entrée change.

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

- **Exemple** :
 - Si $f(x) = x^2$, la dérivée est $f'(x) = 2x$.
 - Interprétation : La pente de la tangente à la courbe en un point donné.
- **Application en Optimisation** :
 - Les dérivées sont utilisées pour trouver les points critiques d'une fonction, où la pente est nulle ($f'(x) = 0$).
 - **Exemple** : Trouver les minima et maxima d'une fonction de coût en Machine Learning.

Mathématiques pour ML

Calcul Différentiel et Intégral

- **Gradient** : est une généralisation de la dérivée pour les fonctions à plusieurs variables. Il indique la direction de la plus forte augmentation d'une fonction.

Gradient Descent

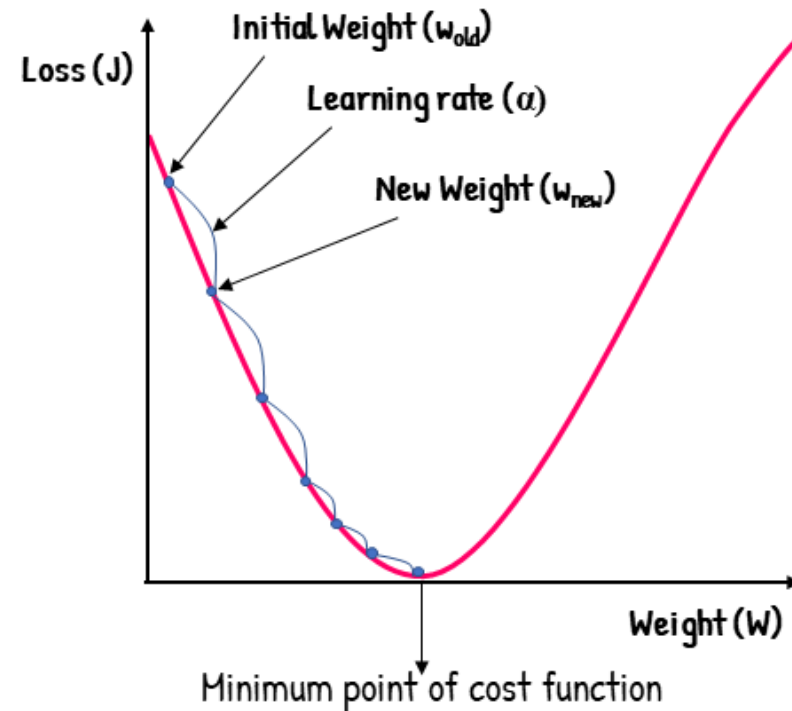
- **Exemple** : si $f(x, y)$

- **Descente de Gradient** dans la direction opposée au gradient.

- **Formule de mise à jour**

- Où α est le taux d'apprentissage

- **Exemple** : Optimisation



$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\delta J}{\delta w}$$

Mathématiques pour ML

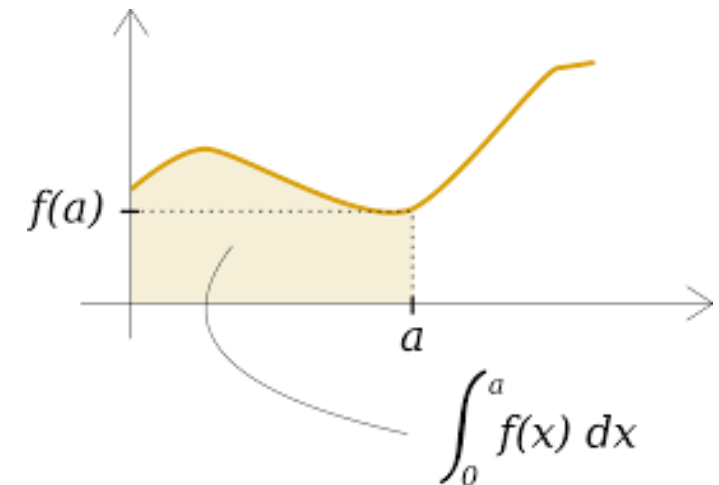
Calcul Différentiel et Intégral

- **Intégrale** : d'une fonction représente l'aire sous la courbe de cette fonction sur un intervalle donné.

$$\int_a^b f(x) dx$$

- **Exemple** : Si $f(x) = x^2$, l'intégrale entre 0 et 2 est :

$$\int_0^2 x^2 dx = \left[\frac{x^3}{3} \right]_0^2 = \frac{8}{3}$$



- **Application** :
 - Utilisée pour calculer les aires, les volumes, et pour trouver la probabilité cumulée dans les distributions continues.
- **Exemple en Machine Learning** : Calculer les surfaces sous les courbes ROC ou PR pour évaluer les performances des modèles.

Python pour le ML

- **Jupyter Notebook** : Environnement interactif permettant de combiner du code, du texte, des visualisations et des équations mathématiques dans un même document.
- **Avantages :**
 - Exécution de code par cellules.
 - Visualisation immédiate des résultats.
 - Facilité de documentation et de partage.
- **Anaconda** : Distribution Python qui inclut un ensemble d'outils pour la science des données, dont Jupyter Notebook.
- **Avantages :**
 - Installation et gestion faciles des bibliothèques.
 - Création d'environnements virtuels pour gérer les dépendances de projet.

Python pour le ML

Bibliothèque

- **NumPy** : Bibliothèque pour le calcul numérique en Python.
- **Pandas** : Bibliothèque pour la manipulation et l'analyse des données.
- **Matplotlib** : Bibliothèque pour la création de graphiques.
- **Scikit-learn** : Bibliothèque pour le machine learning, incluant des algorithmes de classification, régression, clustering.

Prétraitement des Données

Importation

- **Chargement de données à partir de fichiers CSV** : Utilisation de **Pandas** pour importer des datasets.

```
import pandas as pd
# Chargement d'un fichier CSV
df = pd.read_csv('data.csv')
print(df.head())
```

- **Chargement de données à partir de bases de données** : Connexion à des bases de données SQL.

```
import pandas as pd
import sqlite3

# Connexion à la base de données
conn = sqlite3.connect('database.db')
df = pd.read_sql_query('SELECT * FROM table_name', conn)
print(df.head())
```

Prétraitement des Données

Nettoyage

- **Gestion des valeurs manquantes :**

- Suppression des valeurs manquantes

```
# Suppression des lignes avec des valeurs manquantes  
df_clean = df.dropna()  
print(df_clean.head())
```

- Imputation des valeurs manquantes

```
# Remplacer les valeurs manquantes par la moyenne de la colonne  
df['colonne'] = df['colonne'].fillna(df['colonne'].mean())  
print(df.head())
```

Prétraitement des Données

- **Encodage des catégories :**

- Encodage One-Hot:

```
# Encodage One-Hot
df_encoded = pd.get_dummies(df, columns=['cat_col'])
print(df_encoded.head())
```

- Encodage par Label :

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['cat_col'] = le.fit_transform(df['cat_col'])
print(df.head())
```

Prétraitement des Données

Visualisation

- **Matplotlib** : Exemple de graphique en ligne

```
import matplotlib.pyplot as plt
df['colonne'].plot(kind='line')
plt.title('Graphique en Ligne')
plt.show()
```

- **Seaborn** :

- Exemple de graphique de distribution :

```
import seaborn as sns
sns.distplot(df['colonne'])
plt.title('Distribution des Données')
plt.show()
```

- Graphique de corrélation

```
import seaborn as sns
# Matrice de corrélation
corr = df.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Matrice de Corrélation')
plt.show()
```

Concepts Clés du ML

Fonction de Coût : Mesure de la Performance du Modèle

- **Définition :**

- La fonction de coût mesure l'erreur entre les prédictions du modèle et les valeurs réelles.
- Utilisée pour guider l'optimisation du modèle pendant l'apprentissage.

- **Exemples de Fonctions de Coût :**

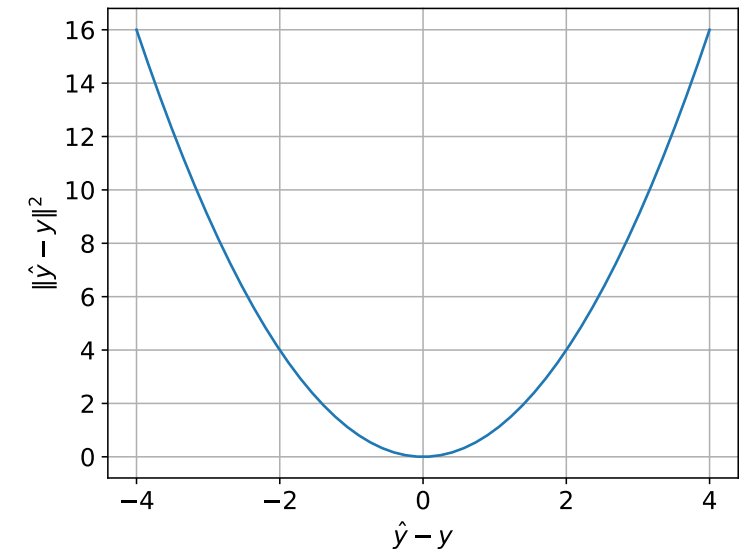
- **Moyenne des erreurs quadratiques (MSE) :**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Où y_i est la valeur réelle, $\hat{y}_i$ la prédiction, et n le nombre d'exemples.

- **Log Loss (pour la classification) :**

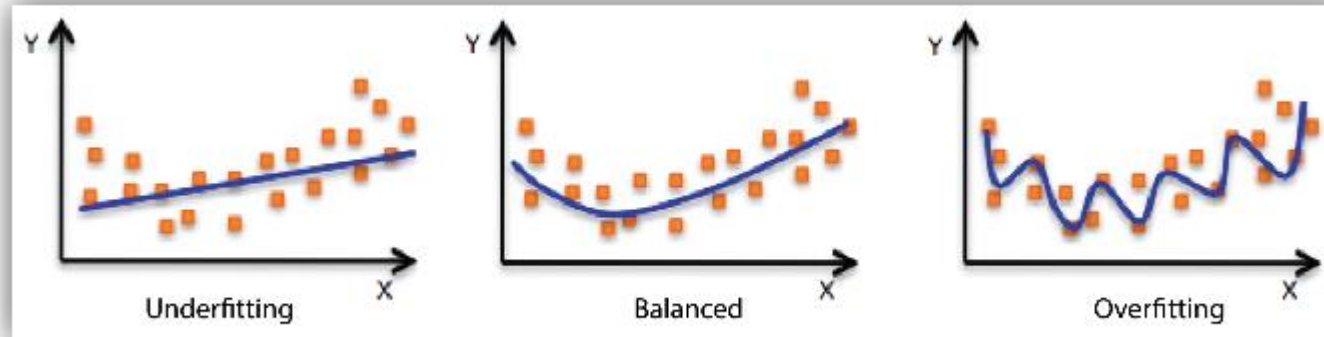
$$LogLoss = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$



Concepts Clés du ML

Surapprentissage vs Sous-apprentissage

- **Surapprentissage (Overfitting) :**
 - Le modèle est trop complexe et s'adapte trop bien aux données d'entraînement, mais généralise mal sur les nouvelles données.
- **Symptômes :** Erreur faible sur les données d'entraînement mais élevée sur les données de test.
- **Sous-apprentissage (Underfitting) :**
 - Le modèle est trop simple pour capturer les tendances des données.
- **Symptômes :** Erreur élevée à la fois sur les données d'entraînement et de test.



Concepts Clés du ML

Évaluation

- **Précision (Precision)** : La proportion de prédictions positives correctes parmi toutes les prédictions positives.

$$Precision = \frac{TP}{TP + FP}$$

Où TP est le nombre de vrais positifs, FP le nombre de faux positifs.

- **Rappel (Recall)** :
- La proportion de vrais positifs détectés parmi tous les échantillons réellement positifs.

$$Recall = \frac{TP}{TP + FN}$$

Où FN est le nombre de faux négatifs

- **F1-Score** : La moyenne harmonique de la précision et du rappel, utile pour évaluer un modèle lorsque les classes sont déséquilibrées.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Concepts Clés du ML

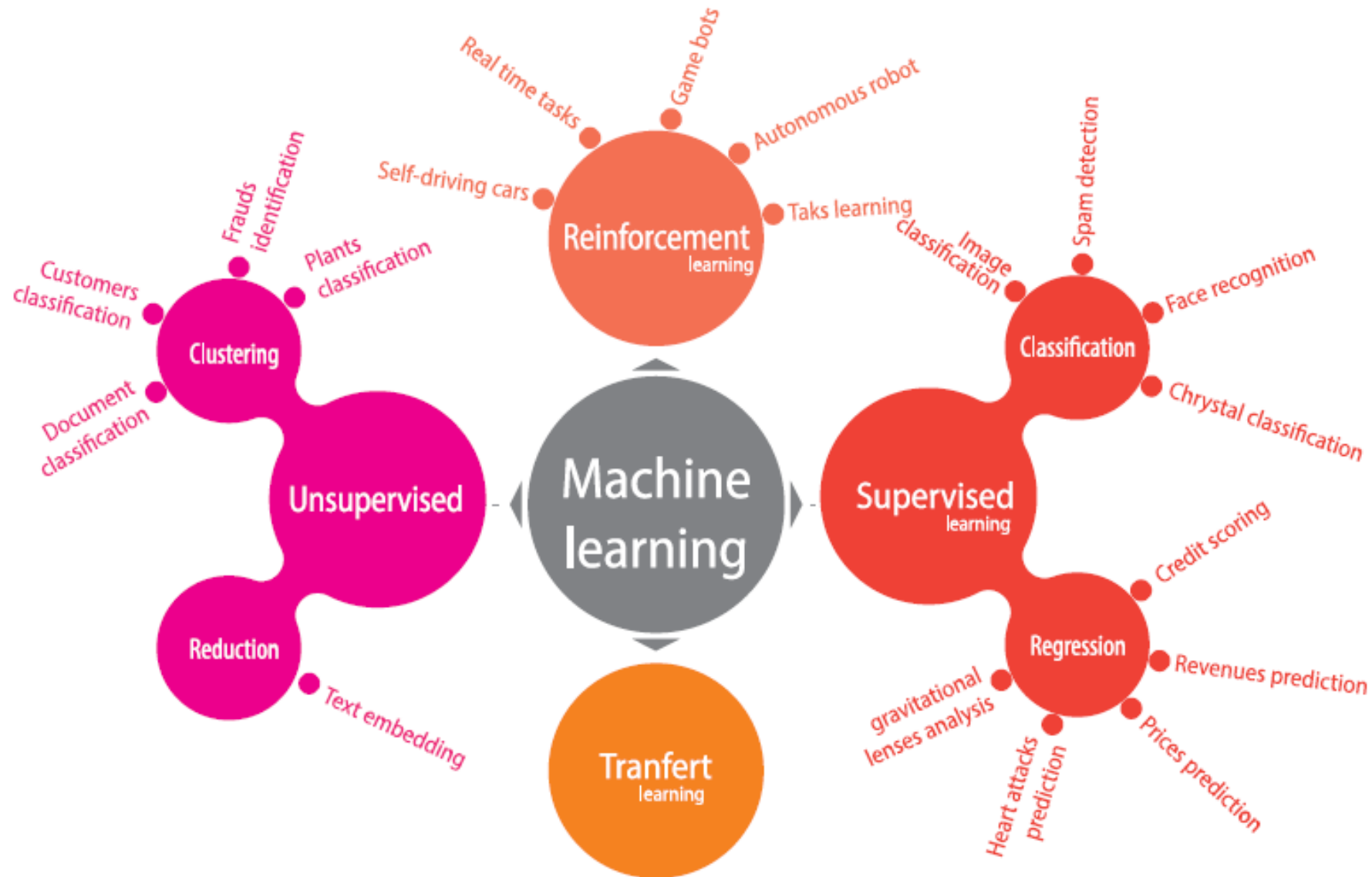
Tableau de confusion

- Exemple

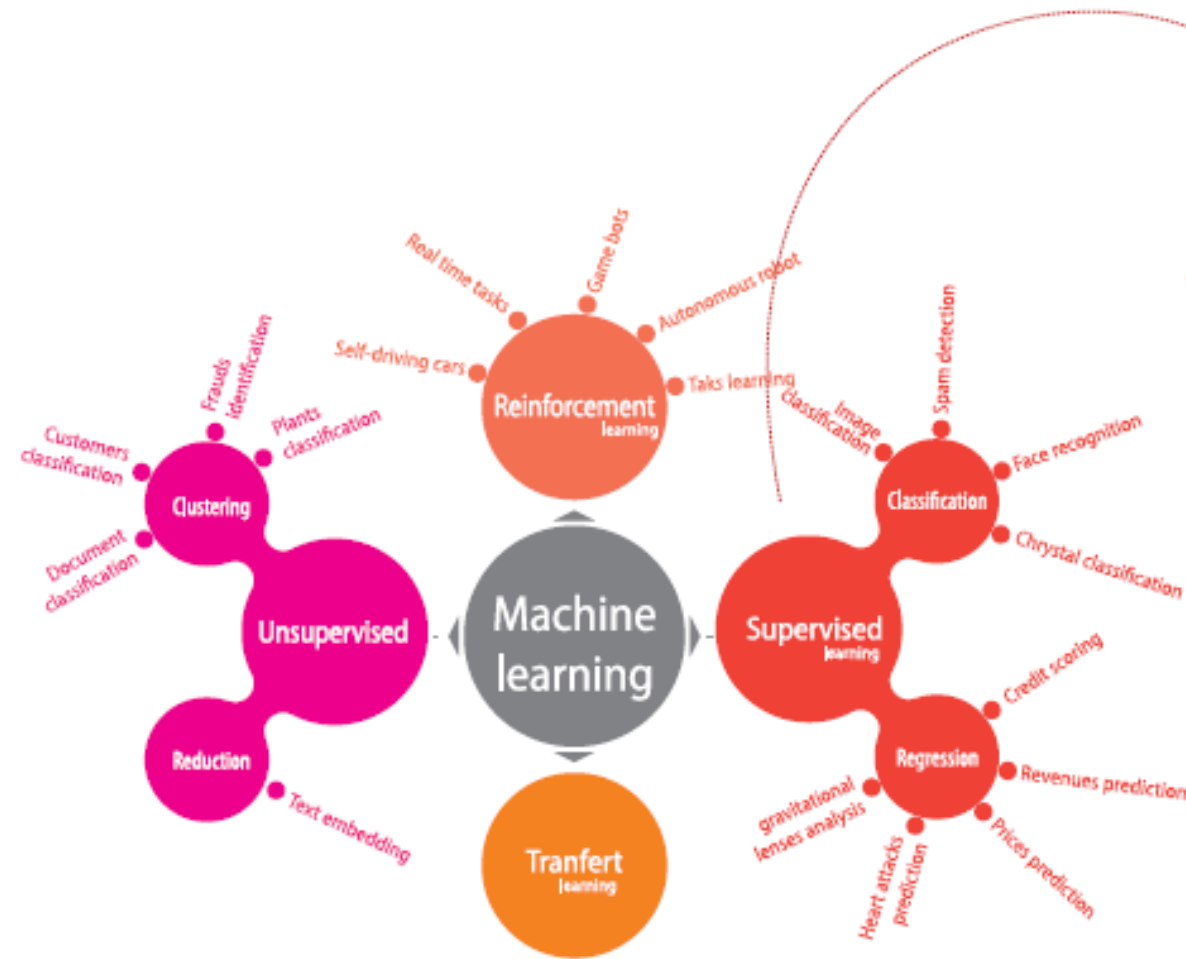
Confusion matrix		Reality	
		Negative : 0	Positive : 1
Prediction	Negative : 0	True Negative : TN	False Negative : FN
	Positive : 1	False Positive : FP	True Positive : TP

L'apprentissage Automatique

Apprentissage Automatique



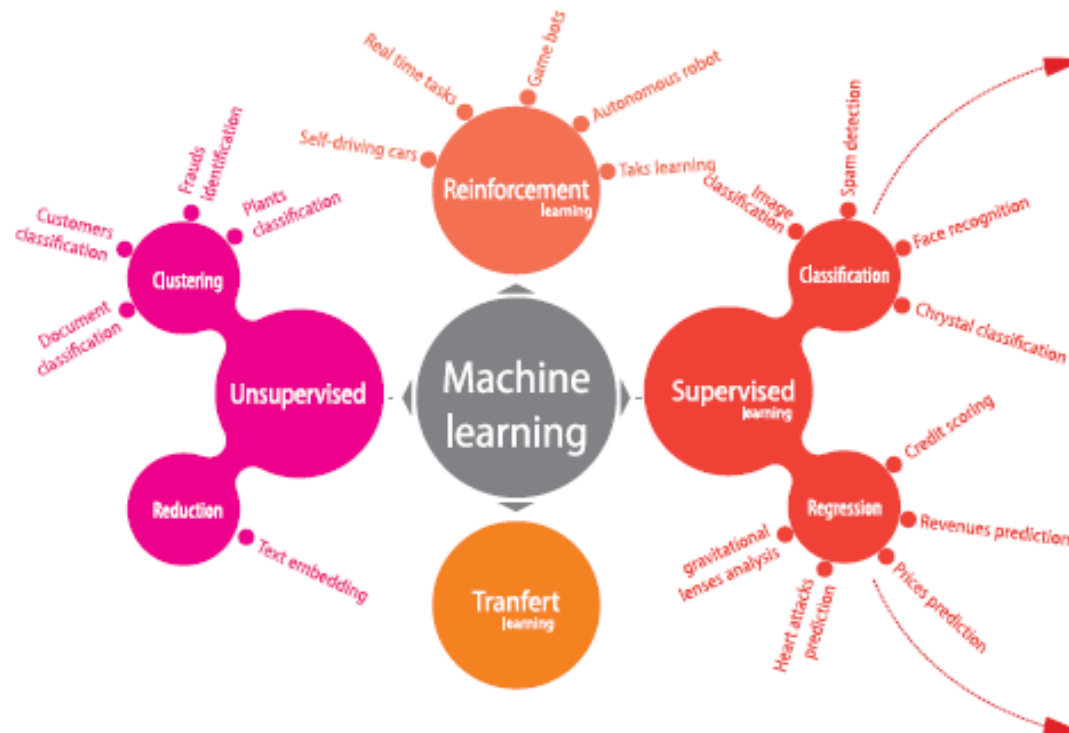
Apprentissage Automatique



Learning
from **labeled data**

Apprentissage Automatique

Learning from labeled data



Classification :

Predict qualitative informations



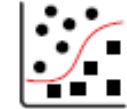
This is a cat



This is a rabbit



Tell me,
what is it ?



Régression :

Predict quantitative informations



150 K€



400 K€



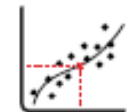
120 K€



100 K€

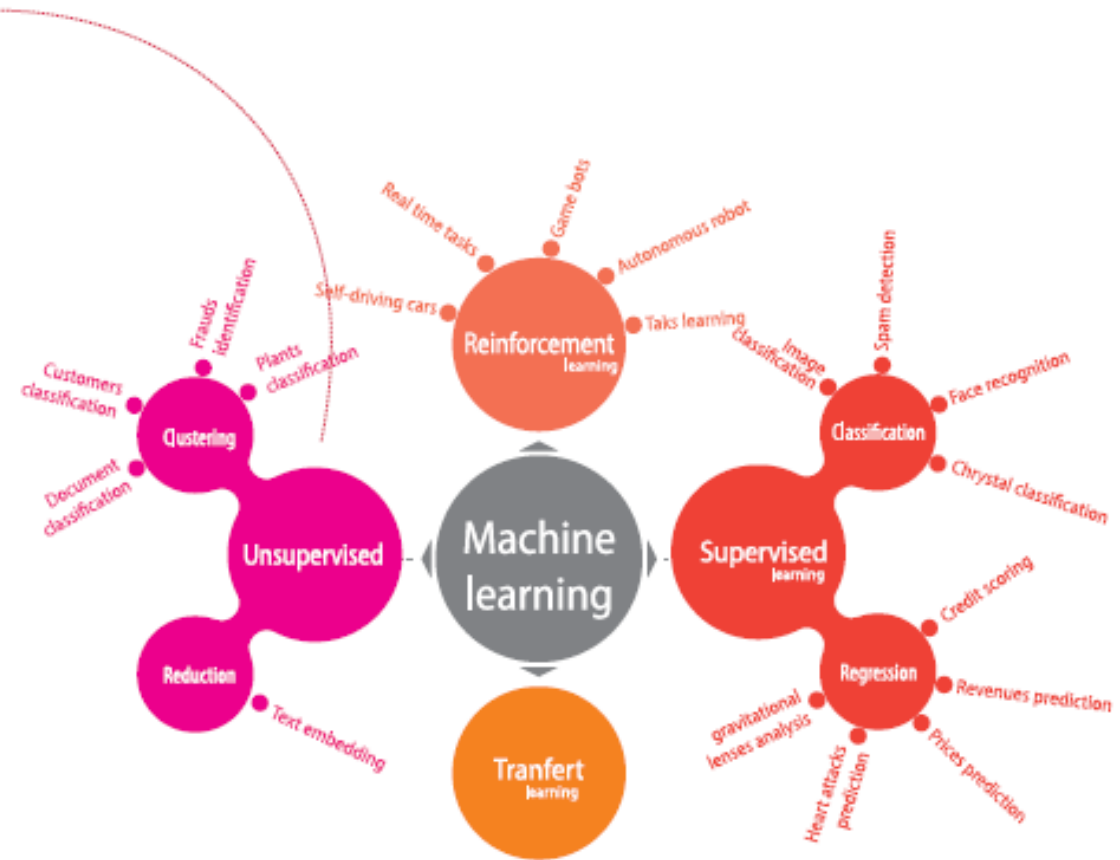


Tell me,
what's the
price ?

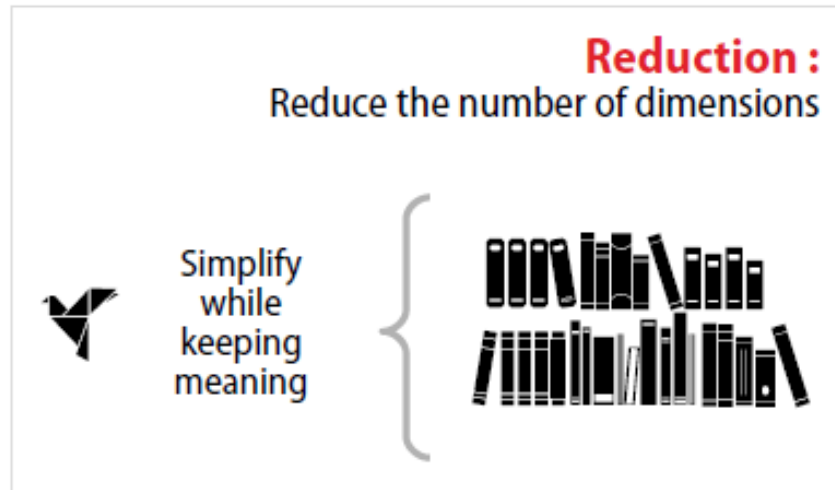


Apprentissage Automatique

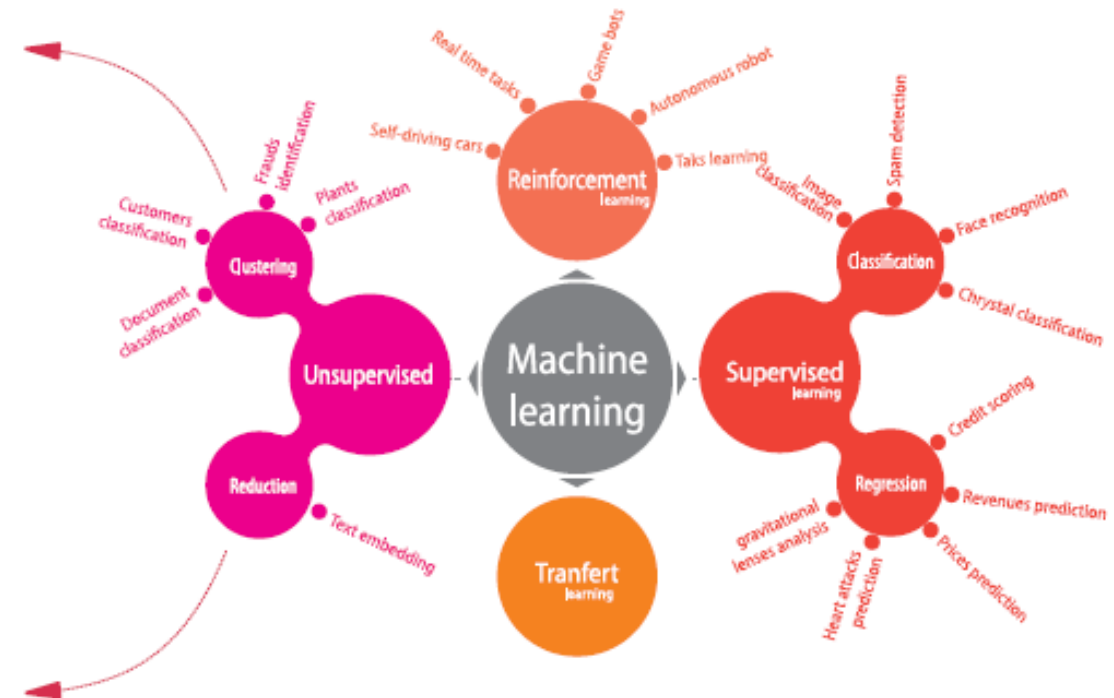
Learning
from data **alone**



Apprentissage Automatique

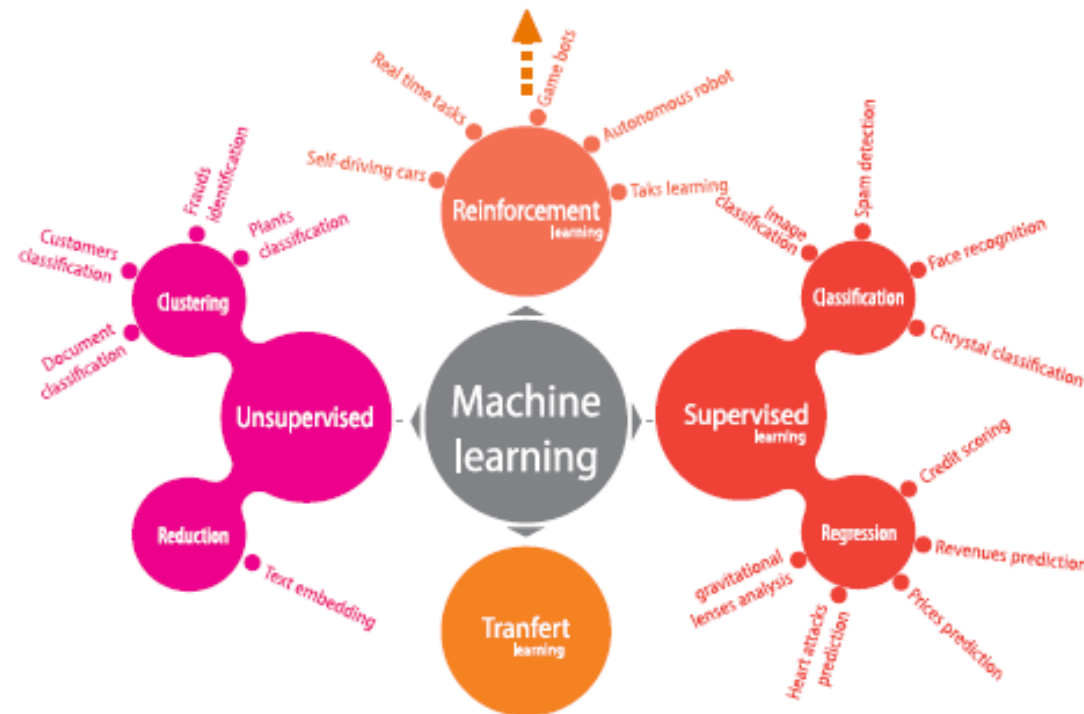
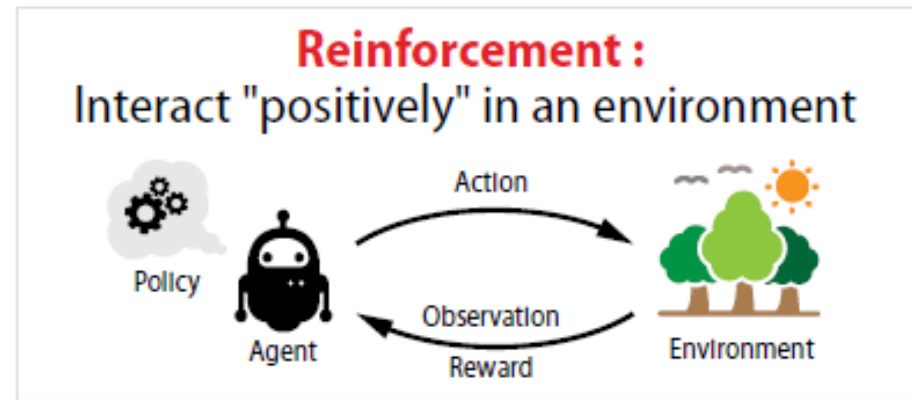


Learning from data alone

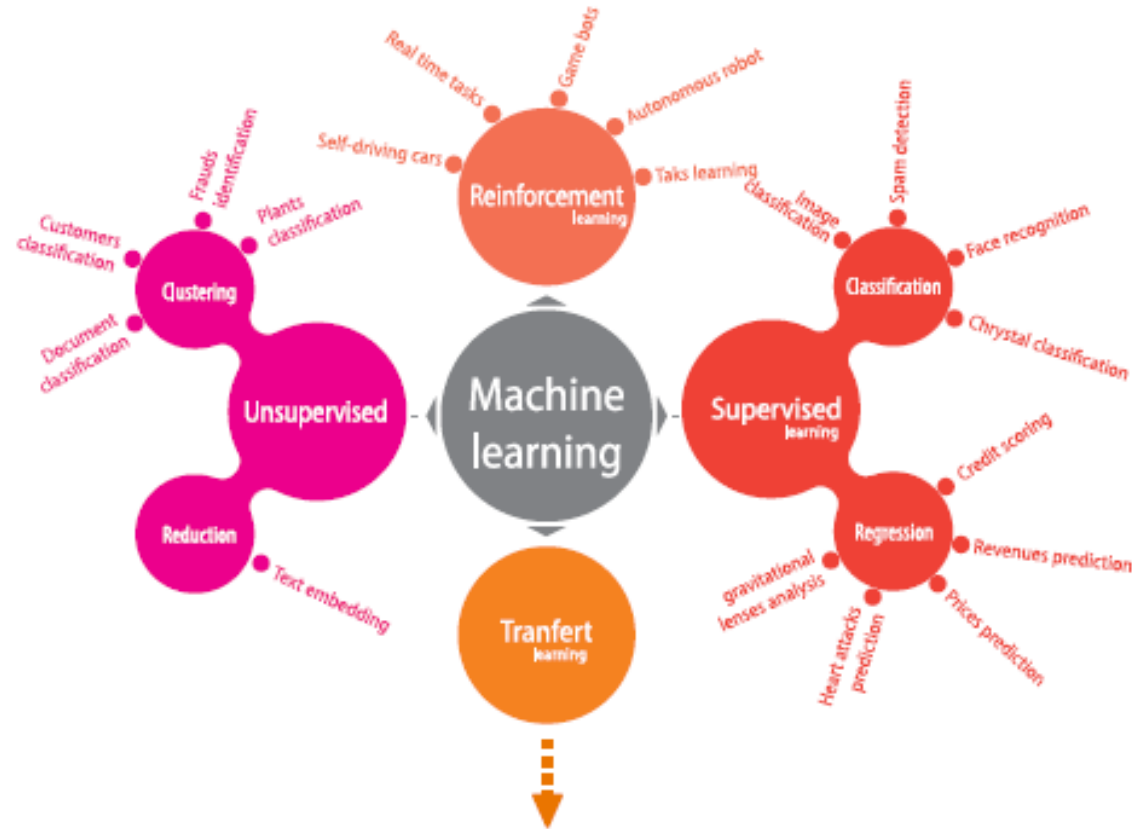


Apprentissage Automatique

Learning from an environment



Apprentissage Automatique



Transfer learning :
Extend, complete or specialize a training

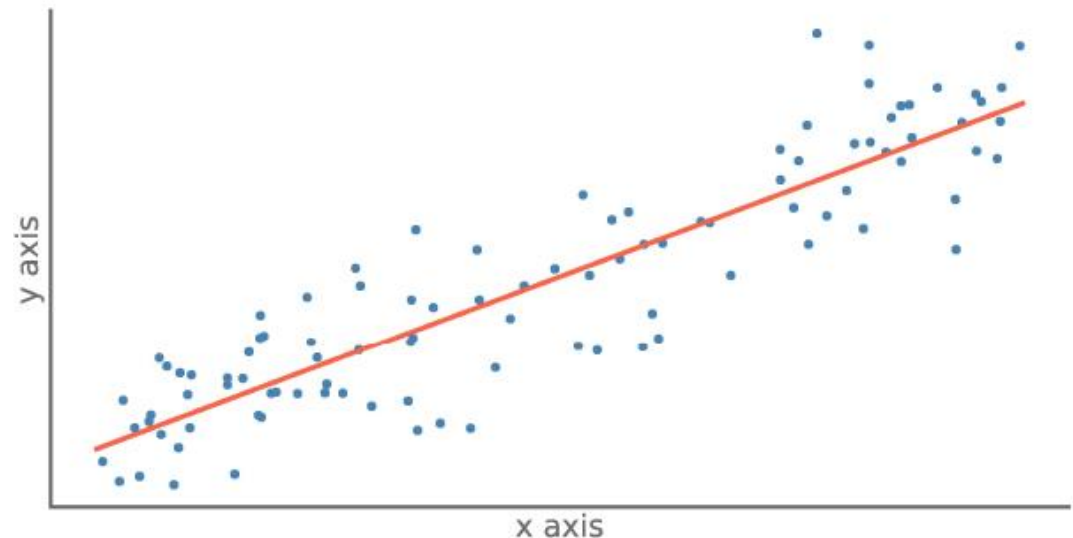
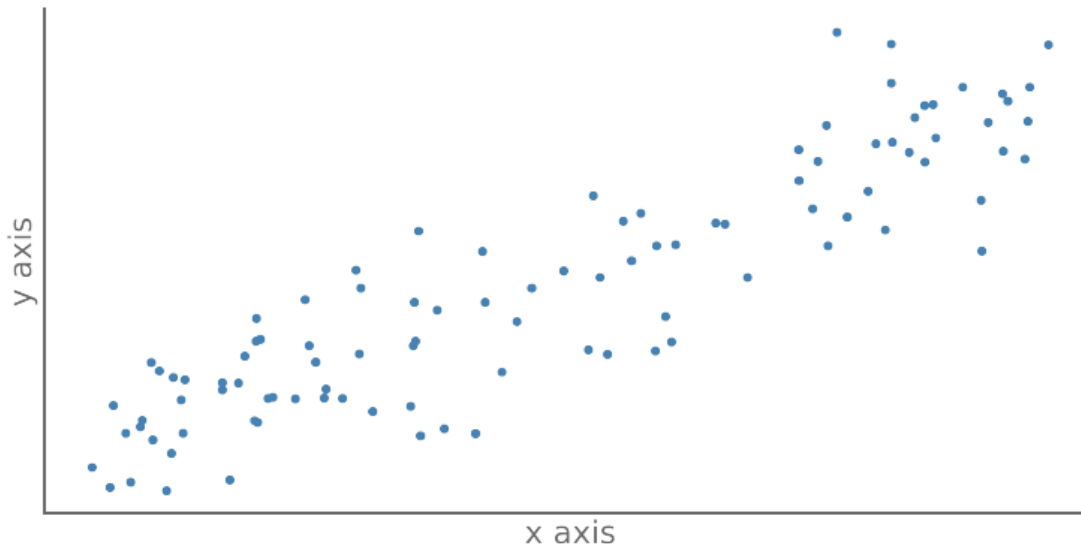


Apprentissage Supervisé

Régression Linéaire

Qu'est-ce que la Régression Linéaire ?

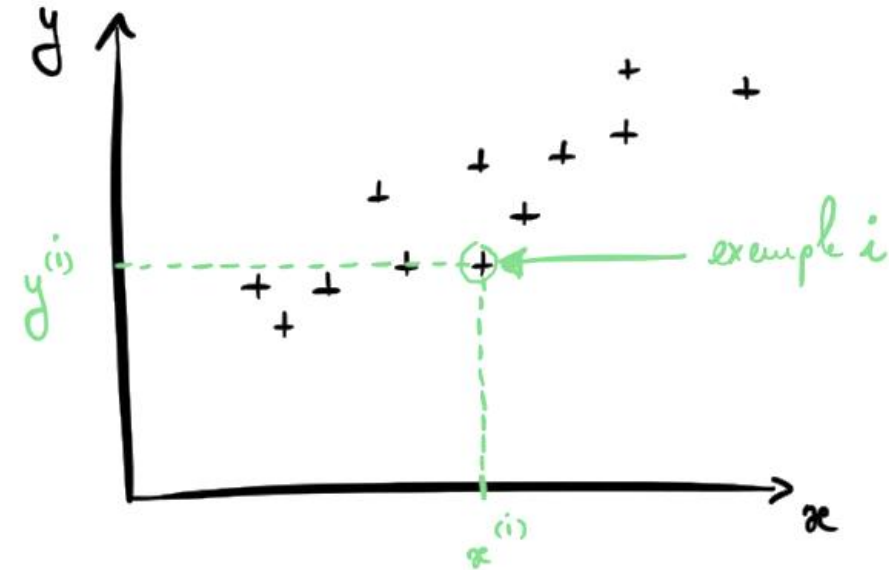
- **Définition** : Technique statistique pour modéliser la relation entre une variable dépendante Y et une ou plusieurs variables indépendantes X .
- Cet algorithme cherche à établir, sous forme d'une droite, une relation entre une variable expliquée et une variable explicative. Par exemple, prédire une note à un examen (variable expliquée) en fonction du nombre d'heures de révisions (variable explicative).
- En d'autres termes, les données d'une série d'observations sont représentées sous forme d'un nuage de points et l'on cherche à trouver une droite passant au plus près de ces points.



Régression Linéaire

Récolter les données

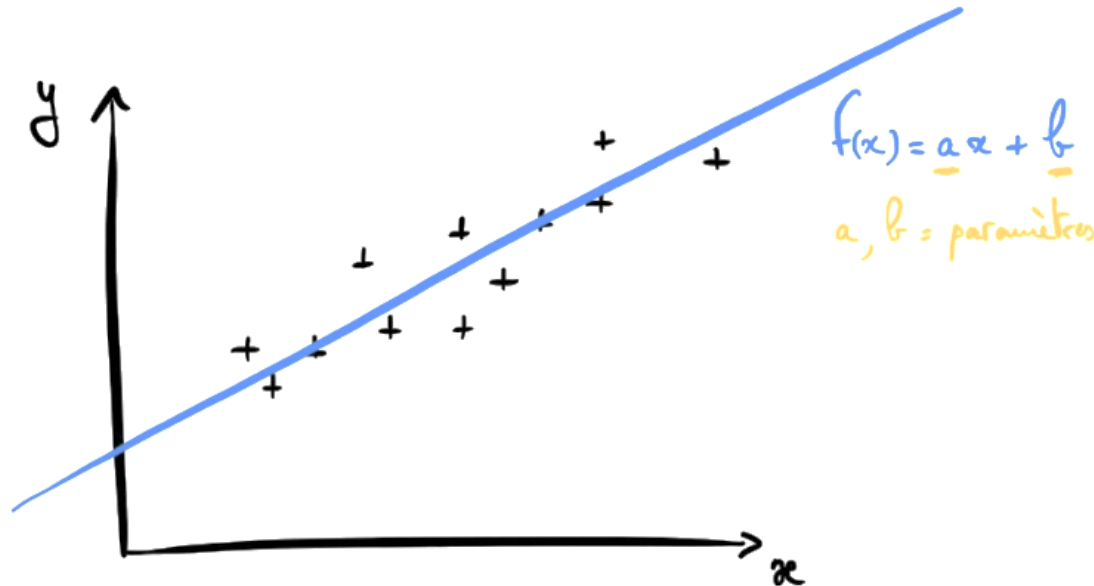
- Imaginez que plusieurs agences immobilières vous aient fourni des données sur des appartements à vendre, notamment le prix de l'appartement (y) et la surface habitable (x). En Machine Learning, on dit que vous disposez de m exemples d'appartements.
- On désigne :
 - $x(i)$ la surface habitable de l'exemple i
 - $y(i)$ le prix de l'exemple i
- En visualisant votre **Dataset**, vous obtenez le nuage de points à droite :



Régression Linéaire

Créer un modèle linéaire

- A partir de ces données, on développe un **modèle linéaire** $f(x)=ax+b$
 - où a et b sont les **paramètres** du modèle.
- Un bon modèle donne de **petites** erreurs entre ses prédictions $f(x)$ et les exemples (y) du Dataset.
- Nous ne connaissons pas les valeurs des paramètres a et b , ce sera le rôle de la machine de les trouver, de sorte à tracer un modèle qui s'insère bien dans notre nuage de point comme ci-dessous :



Régression Linéaire

La Fonction Coût

- Pour la régression linéaire, on utilise la **norme euclidienne** pour mesurer les **erreurs** entre $f(x)$ et (y) .
- la formule pour exprimer l'erreur i entre le prix $y^{(i)}$ et la prédiction faites en utilisant la surface $x^{(i)}$:

$$erreur^{(i)} = (f(x^{(i)}) - y^{(i)})^2$$

- Exemple :
 - imaginez que le 10^{ième} exemple de votre Dataset soit un appartement de $x^{(10)} = 80 m^2$ dont le prix s'élève à $y^{(10)} = 1000,000$ DH et que votre modèle prédise un prix de $f(x^{(10)}) = 1000,002$ DH.
 - L'erreur pour cet exemple est donc :
 - $erreur^{(10)} = (f(x^{(10)}) - y^{(10)})^2$
 - $erreur^{(10)} = (100,002 - 100,000)^2$
 - $erreur^{(10)} = (2)^2$
 - $erreur^{(10)} = 4$

Fonction Coût (MSE - Mean Squared Error)

- La fonction coût est utilisée pour évaluer la performance du modèle de régression en mesurant l'erreur entre les prédictions et les valeurs réelles. La fonction coût la plus couramment utilisée pour la régression linéaire est l'Erreur Quadratique Moyenne (MSE) :

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- y_i : Valeur réelle de la variable dépendante pour l'observation i .
 - $\hat{y}_i$: Valeur prédite par le modèle pour l'observation i .
 - n : Nombre total d'observations.
- Objectif** : Minimiser le MSE pour améliorer la qualité du modèle. Un MSE plus faible indique une meilleure adéquation entre les prédictions et les données réelles.

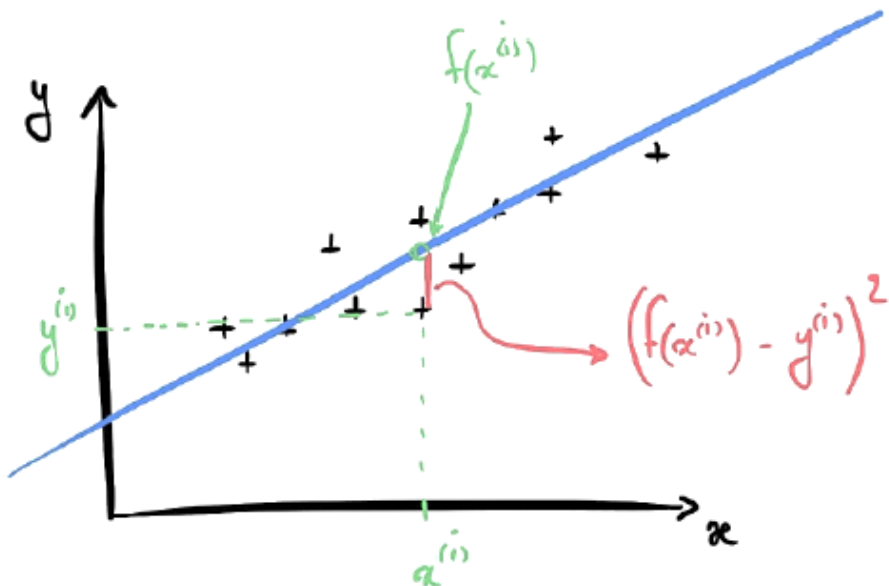
Régression Linéaire

La Fonction Coût

- Chaque prédiction s'accompagne d'une erreur, on a donc **m erreurs**.
- On définit la **Fonction Coût $J(a,b)$** comme étant la **moyenne** de toutes les erreurs :

$$J(a,b) = \frac{1}{2m} \sum_{i=1}^m \text{erreur}^i$$

$$J(a,b) = \frac{1}{2m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)})^2$$



$$J(a,b) = \frac{1}{2m} \sum_{i=1}^m (\underbrace{ax^{(i)} + b}_{f(x^{(i)})} - y^{(i)})^2$$

Fonction Coût.

Régression Linéaire

Trouver les paramètres qui minimisent la Fonction Coût

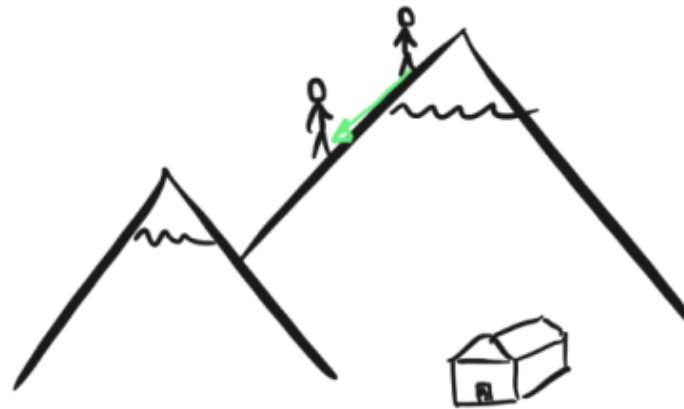
- La prochaine étape est de laisser la machine apprendre quels sont les paramètres qui **minimisent** la Fonction Coût, c'est-à-dire les paramètres qui nous donnent le meilleur modèle.
- Pour trouver le minimum, on utilise un algorithme d'optimisation qui s'appelle **Gradient Descent** (la descente de gradient).
- Le principe :
- Imaginez-vous perdu en montagne. Votre but est de rejoindre le refuge qui se trouve au point le **plus bas** de la vallée et vous n'avez pas de carte. Pour vous en sortir, voici une stratégie à adopter :
 - Depuis votre position actuelle, vous partez en direction de là où la **pente descend** le plus **fort**.
 - Vous avancez une **certaine distance** en suivant cette direction (même si ça implique de remonter une pente)
 - Une fois cette distance parcourue, vous répétez les 2 premières opérations en boucle, jusqu'à atteindre le point le plus bas de la vallée.

Régression Linéaire

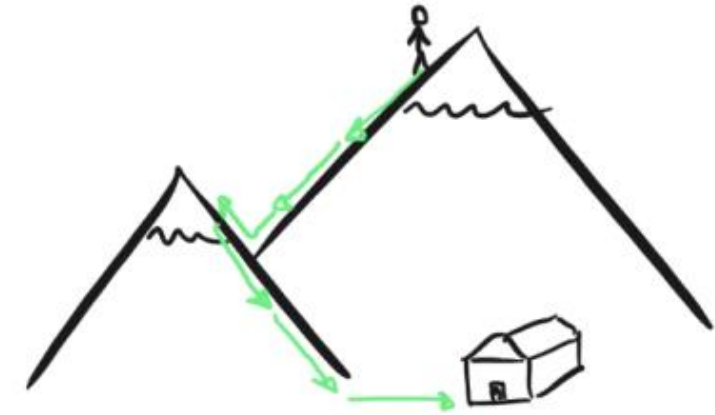
Trouver les paramètres qui minimisent la Fonction Coût



Etape 1: Trouver la pente la plus forte



Etape 2: Marcher une certaine distance dans cette direction



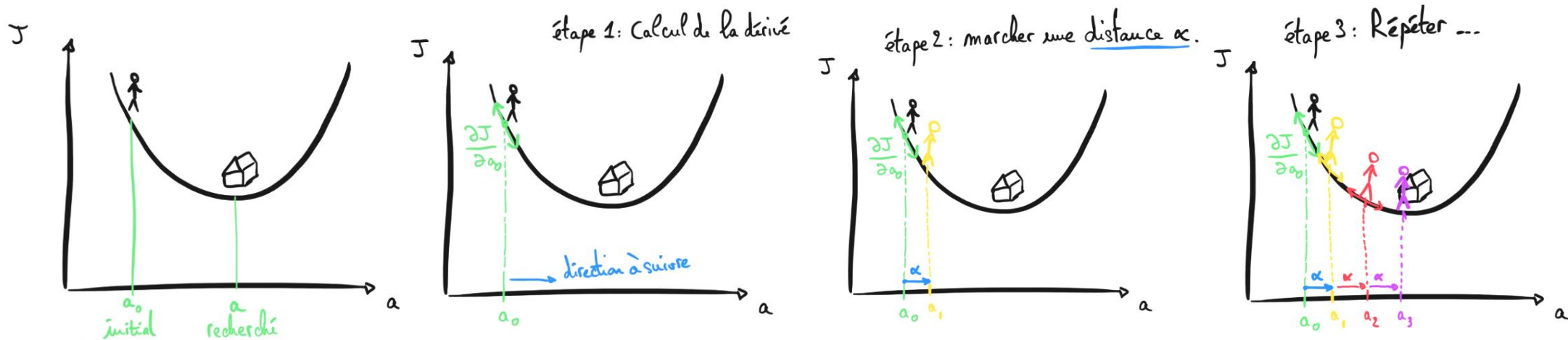
Etape 3: Répéter les étapes 1 et 2 en boucle

-
- Les étapes 1, 2 et 3 forment ce qu'on appelle l'algorithme de **Gradient Descent**.

Régression Linéaire

Trouver les paramètres qui minimisent la Fonction Coût

- Cet algorithme vous permet de trouver le **minimum** de la Fonction Coût $J(a, b)$ (le point le plus bas de la montagne) en partant de coordonnées a et b **aléatoires** (votre position initiale dans la montagne) :
 - Calculer la **pente** de la Fonction Coût, c'est-à-dire la **dérivée** de $J(a, b)$.
 - **Evoluer** d'une certaine **distance** α dans la direction de la pente la plus forte. Cela a pour résultat de modifier les paramètres a et b
 - Recommencer les étapes 1 et 2 jusqu'à atteindre le minimum de $J(a, b)$.



Régression Linéaire

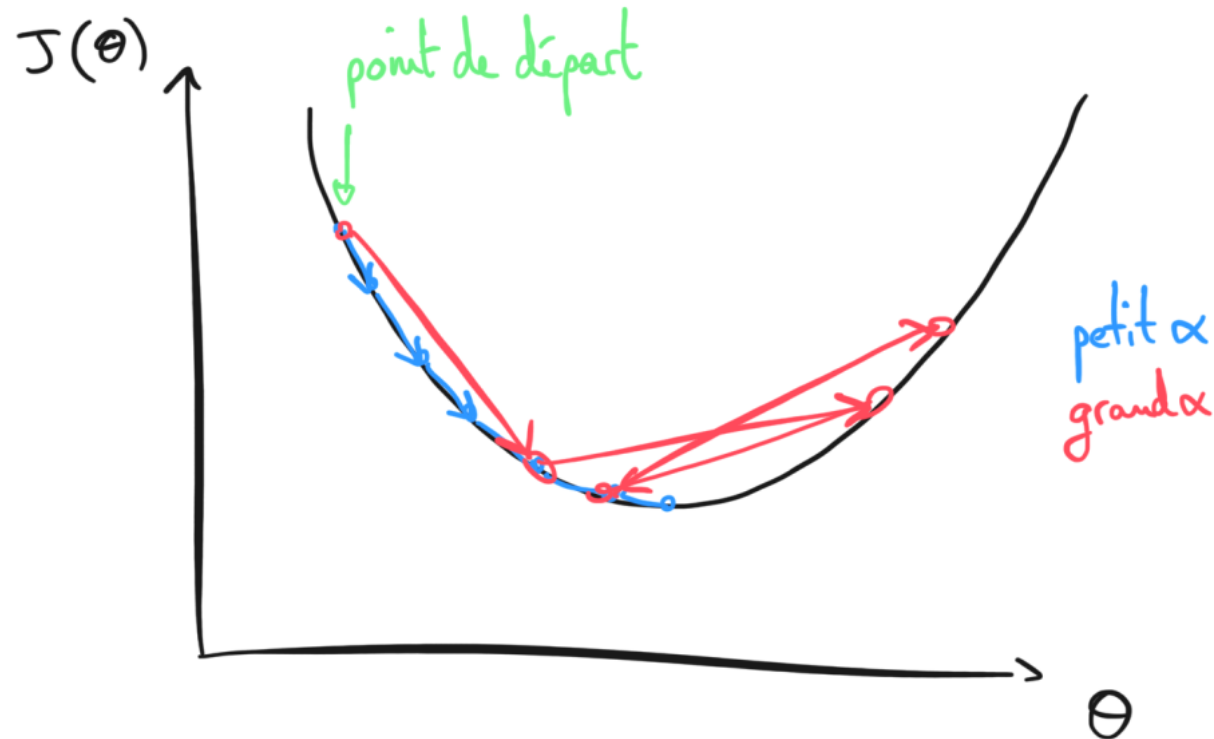
Trouver les paramètres qui minimisent la Fonction Coût

- L'objectif donc est de trouver les paramètres a et b qui **minimisent** $J(a, b)$.
- Pour cela, nous allons choisir a et b au **hasard** (nous allons nous perdre en montagne) puis allons utiliser **en boucle** la descente de gradient pour mettre à jour nos paramètres dans la direction de la **Fonction Coût** la **plus faible**.
- *Répéter en boucle:*
 - $a = a - \alpha \partial J(a, b) / \partial a$
 - $b = b - \alpha \partial J(a, b) / \partial b$
- à chaque itération de cette boucle, les paramètres a et b sont mis à jour en **soustrayant** leur propre valeur à la valeur de la **pente** $\partial J(a, b) / \partial \dots$ multipliée par la distance à parcourir α .
- On appelle α la vitesse d'apprentissage (**Learning rate**).

Régression Linéaire

Trouver les paramètres qui minimisent la Fonction Coût

- Si la vitesse est trop lente, le modèle peut mettre longtemps à être entraîné, mais si la vitesse est trop grande, alors la distance parcourue est trop longue et le modèle peut ne jamais converger. Il est important de trouver un juste milieu. Le dessin ci-dessous illustre mes propos.



Régression Linéaire

Calcul des dérivées partielles

- Pour implémenter l'algorithme de Gradient Descent, il faut donc calculer les **dérivées partielles** de la Fonction Coût. Rappelez-vous qu'en mathématique, la dérivée d'une fonction en un point nous donne la valeur de sa pente en ce point.

- **Fonction Coût :**
$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m (ax^{(i)} + b - y^{(i)})^2$$

- **Dérivée selon le paramètre a :**
$$\frac{\partial J(a, b)}{\partial a} = \frac{1}{m} \sum_{i=1}^m (ax^{(i)} + b - y^{(i)}) \times x^{(i)}$$

- **Dérivée selon le paramètre b :**
$$\frac{\partial J(a, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (ax^{(i)} + b - y^{(i)})$$

- Sachant que la dérivée d'une **fonction composée** : $(g \circ f)' = f' \times g' \circ f$
- Avec : $f = ax + b - y$ et $g = (f)^2$
- En dérivant, le carré tombe et se simplifie avec la fraction $1/2m$ pour devenir $1/m$ et $x(i)$ apparait en facteur pour la dérivée par rapport à a .

Régression Linéaire

Pourquoi diviser par 2 le MSE

- Dans la descente de gradient, nous mettons à jour les poids w et le biais b selon :

$$w := w - \eta \frac{\partial J}{\partial w}$$

- Calculons la dérivée de $J(w,b)$ par rapport à w :

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i) \cdot (-x_i)$$

- Si nous avons utilisé la **fonction sans division par 2**, la dérivée aurait été

$$\frac{\partial J}{\partial w} = \frac{2}{m} \sum_{i=1}^m (y_i - \hat{y}_i) \cdot (-x_i)$$

- L'ajout du facteur $1/2$ annule le facteur 2 provenant de la dérivation de $(y_i - \hat{y}_i)^2$, ce qui **simplifie le calcul de la mise à jour des poids**.

Régression Polynomiale et multiple

Régression multiple

- Extension de la régression linéaire simple à plusieurs variables explicatives
- Imaginons que chaque entrée $x^{(i)}$ n'est pas un nombre unique, mais un vecteur de
- k nombres $x^{(i1)}, \dots, x^{(ik)}$. Le modèle de régression linéaire multiple fait l'hypothèse que :

$$y_i = a_0 + a_1x_{i,1} + a_2x_{i,2} + \dots + a_px_{i,p} + \varepsilon_i ; i = 1, \dots, n$$

- $(a_0, a_1, \dots, a_p)$ Sont les paramètres (coefficients) du modèle que l'on veut estimer à l'aide des données
- ε quantifie les écarts entre les valeurs réellement observées et les valeurs prédites par le modèle
- **Exemple d'application**
 - Prédire le prix d'une maison en fonction de **plusieurs** caractéristiques : superficie, nombre de chambres, localisation, etc.

Régression multiple

Estimation des coefficients

- Utilisation de la **moindre somme des carrés** (OLS) pour minimiser l'erreur :

$$\min_{\beta} \sum (y_i - \hat{y}_i)^2$$

Régression multiple

Formulation Matricielle du Modèle

- En régression linéaire, nous avons un modèle sous la forme :

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

- On peut réécrire cela sous une **forme matricielle** :

$$\mathbf{Y} = \mathbf{XW}$$

- où :
 - $\mathbf{Y}$ est le vecteur des valeurs cibles (dimension $m \times 1$),
 - $\mathbf{X}$ est la **matrice des caractéristiques** avec une colonne de 1 pour le biais (dimension $m \times (n+1)$),
 - $\mathbf{W}$ est le vecteur des poids (dimension $(n+1) \times 1$).

- Notation Matricielle**

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

Régression multiple

Estimation des Coefficients avec la Formule des Moindres Carrés

- La fonction de coût (erreur quadratique moyenne) est :

$$J(\mathbf{W}) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

- En notation matricielle, l'erreur peut être exprimée comme :

$$J(\mathbf{W}) = \frac{1}{2m} \|\mathbf{Y} - \mathbf{XW}\|^2$$

- Minimiser $J(\mathbf{W})$ revient à résoudre le **problème des moindres carrés**, et la solution analytique est donnée par :

$$\mathbf{W} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

- Cela s'appelle **l'inversion de la matrice normale**, utilisée pour trouver directement les **coefficients optimaux**.

Régression Polynomiale

- **Pourquoi utiliser une régression polynomiale ?**
 - La relation entre x et y n'est pas **linéaire**.
 - La régression multiple ne capture pas des courbures complexes.

- **Forme du modèle**

- Une **régression polynomiale de degré 2** (quadratique) :

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon$$

- Une **régression polynomiale de degré n** :

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n + \epsilon$$

Régression Polynomiale

- Les coefficients sont calculés en minimisant l'erreur quadratique moyenne, en utilisant la **formule des moindres carrés** :

$$\beta = (X^T X)^{-1} X^T Y$$

- où :
- X est la **matrice des variables explicatives** (incluant $x^1, x^2, \dots, x^n$),
- Y est le **vecteur des valeurs cibles**,
- β est le **vecteur des coefficients à estimer**.

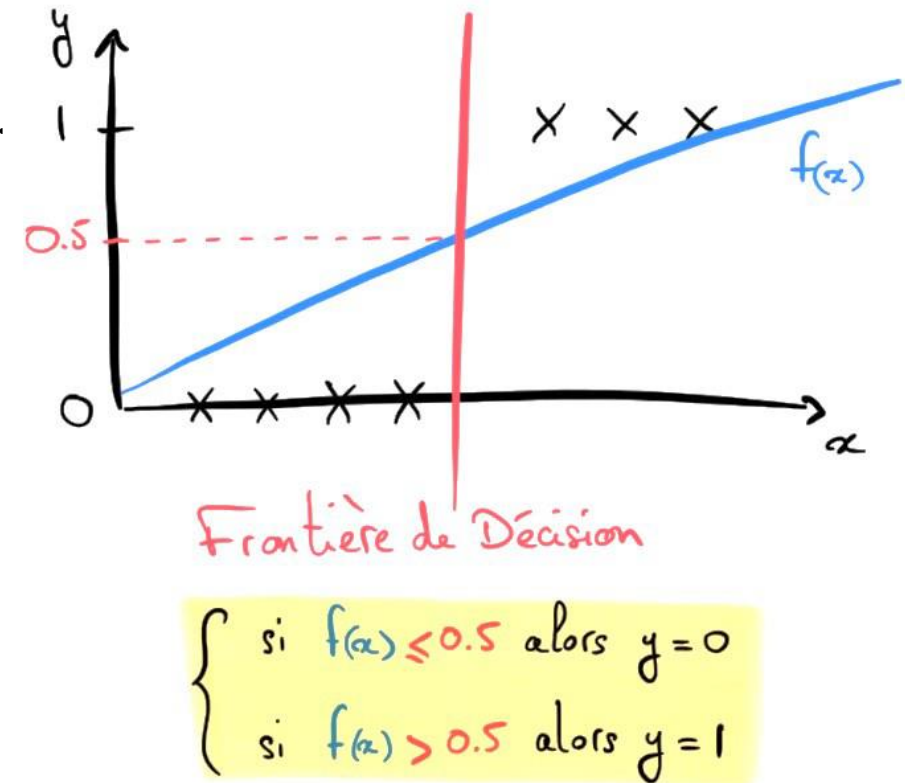
Classification

Régression Logistique

classification

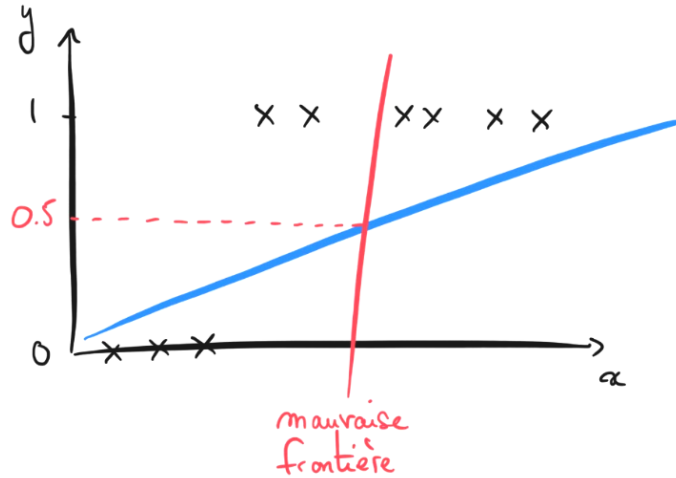
Régression Logistique

- Supposant que je veux classer un email en tant que '**spam**' ou '**non spam**'.
- Dans ce genre de problème, on aura un Dataset contenant une variable target **y** pouvant prendre 2 valeurs seulement, par exemple **0** ou **1**
 - si $y = 0$, alors l'email n'est pas un spam
 - si $y = 1$, alors l'email est un spam
- On dit également que l'on a **2 classes**, c'est une classification **binaire**.
- Pour ces problèmes, on ajoute au modèle une **frontière de décision** qui permet de classer un email dans la *classe* 0 ou la *classe* 1.

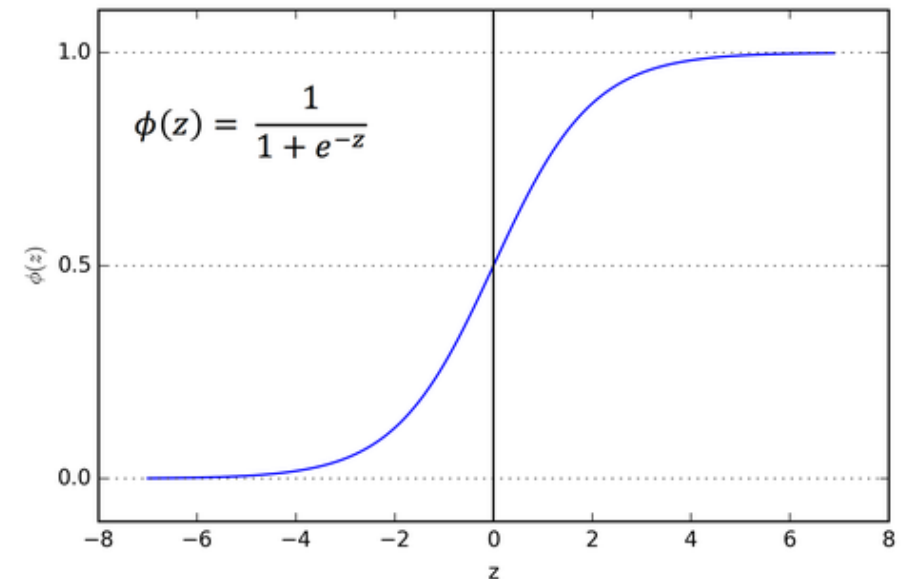


Régression Logistique

- Pour les problèmes de classification binaire, un modèle linéaire $F = X \cdot \theta$ ne convient pas:



- On développe alors une nouvelle fonction pour les problèmes de classification binaire, c'est la **fonction logistique** (aussi appelé fonction **sigmoïde** ou tout simplement sigma σ). Cette fonction a la particularité d'être toujours comprise en 0 et 1.

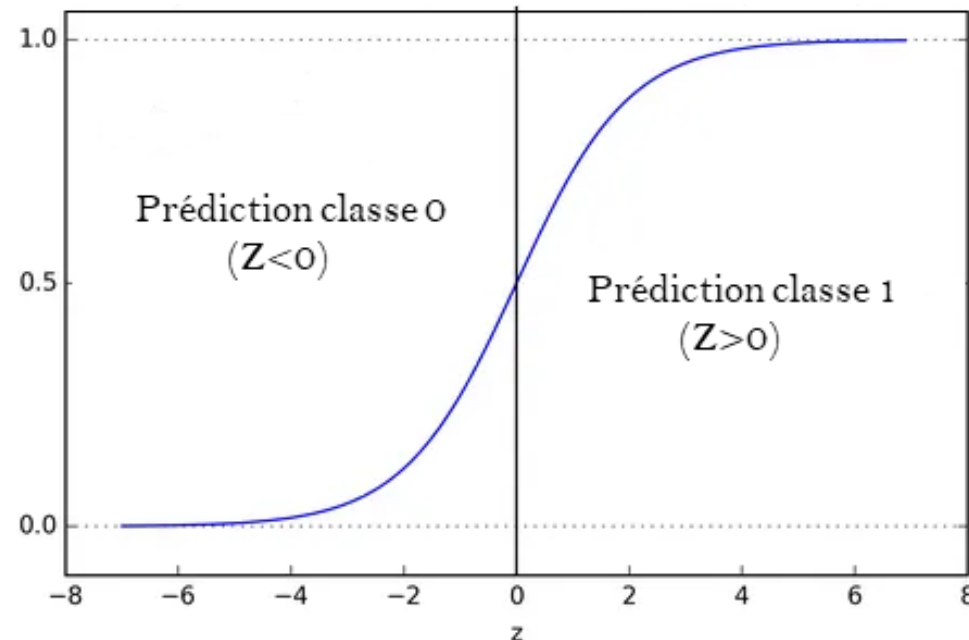


Régression Logistique

- Pour améliorer ce modèle on va accompagner chaque prédiction d'une probabilité, plus une observation sera éloignée de la frontière de décision plus il sera évident qu'elles appartiennent bien à sa classe.
- Pour ce faire, nous pourrions employer une fonction d'activation qui produit une sortie se rapprochant progressivement de zéro ou de un à mesure que l'on s'éloigne de la frontière de décision, là où la fonction f est égale à zéro.
- La fonction qui nous permet de faire ça, c'est la fonction **sigmoïde** également appelé **fonction logistique** dont l'expression est :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

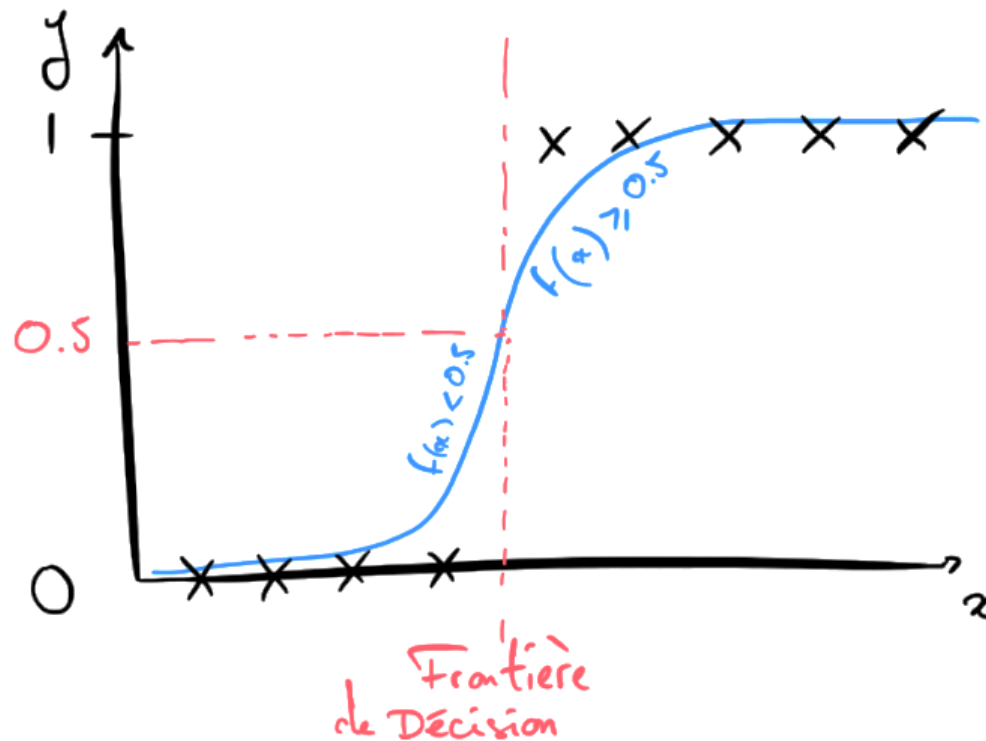
- Cette fonction permet de convertir la sortie z en une probabilité $\sigma(z)$.



Régression Logistique

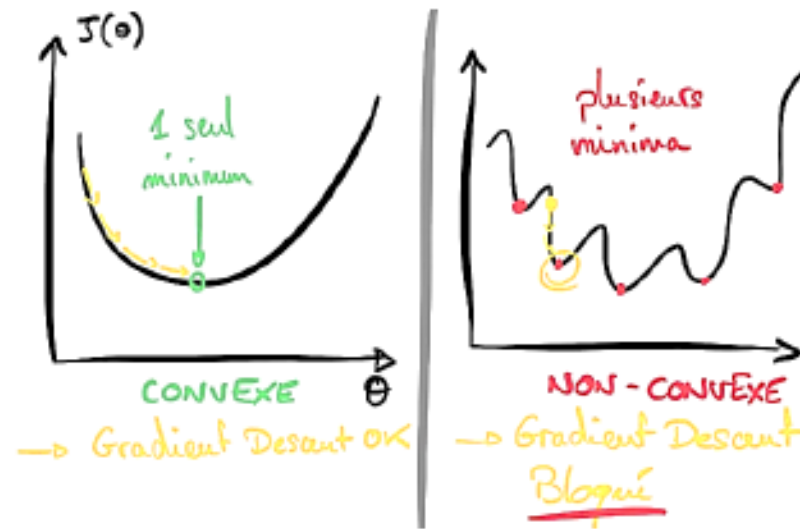
- A partir de cette fonction, il est possible de définir une **frontière** de **décision**. Typiquement, on définit un seuil à **0.5** comme ceci :

$$\begin{cases} y = 0 \text{ si } \sigma(X \cdot \theta) < 0.5 \\ y = 1 \text{ si } \sigma(X \cdot \theta) \geq 0.5 \end{cases}$$



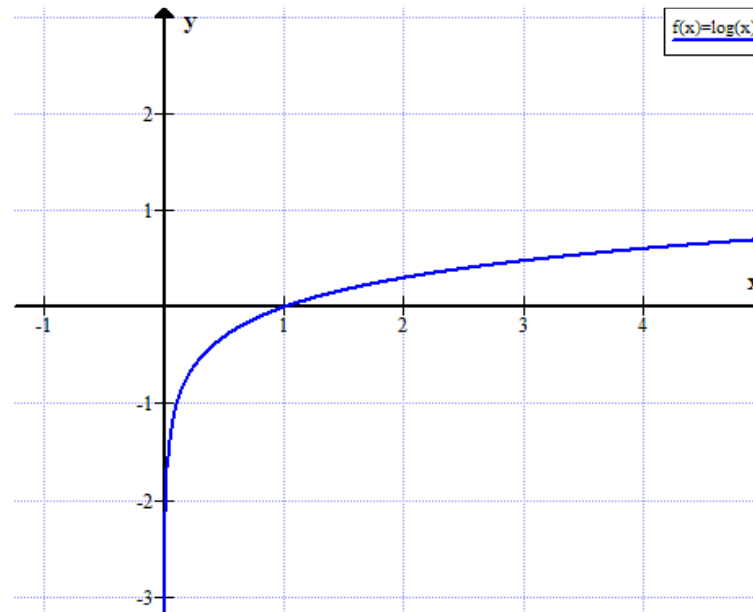
Régression Logistique

- Pour la régression linéaire, la Fonction Coût $J(\theta) = \frac{1}{2m} \sum (X \cdot \theta - Y)^2$ donnait une **courbe convexe** (qui présente un unique minima). C'est ce qui fait que l'algorithme de **Gradient Descent** fonctionne.
- En revanche, utiliser cette fonction pour le **modèle Logistique** ne donnera pas de **courbe convexe** (dû à la non-linéarité) et l'algorithme de **Gradient Descent** se **bloquera** au premier minima rencontré, sans trouver le minimum **global**.



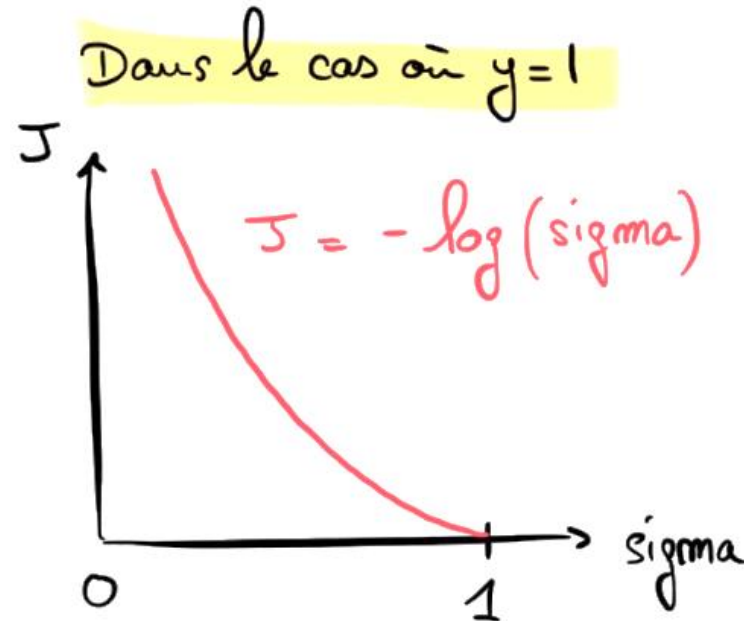
Régression Logistique

- Il faut donc développer une nouvelle Fonction Coût spécialement pour la régression logistique. On utilise alors la fonction **logarithme** pour **transformer** la fonction sigma en une fonction convexe en séparant les cas où $y = 1$ des cas où $y = 0$.



Régression Logistique

- Fonction Coût dans les cas où $y = 1$
- Voici la Fonction Coût que l'on utilise dans les cas où $y = 1$: $J(\theta) = -\log(\sigma(X.\theta))$

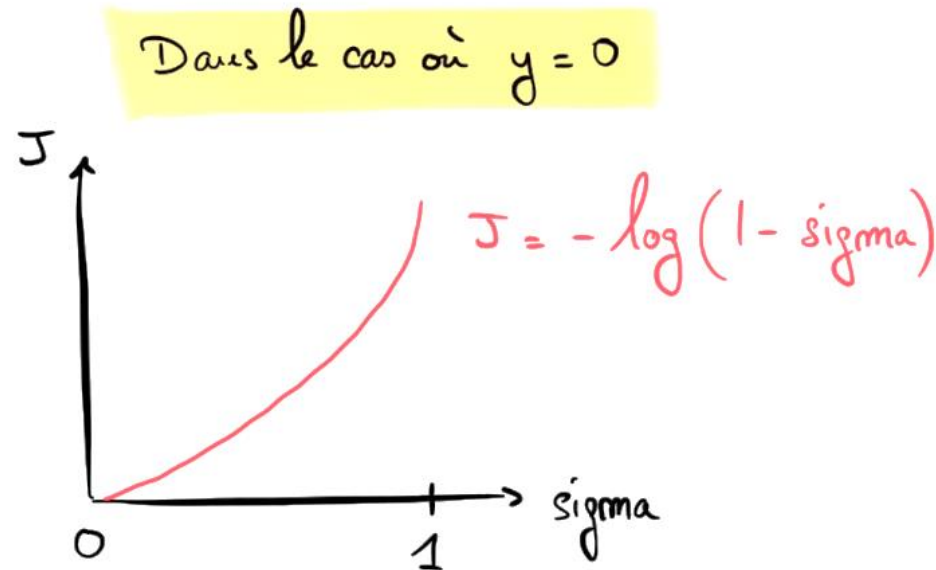


- **Explications :**

Si notre modèle prédit $\sigma(x) = 0$ alors que $y = 1$, on doit pénaliser la machine par une **grande** erreur (un grand coût). La fonction logarithme permet de tracer cette courbe avec une propriété convexe, ce qui poussera le Gradient Descent à trouver les **paramètres** θ pour un coût qui tend vers 0.

Régression Logistique

- Fonction Coût dans les cas où $y = 0$
- Voici la Fonction Coût que l'on utilise dans les cas où $y = 0$: $J(\theta) = -\log(1 - \sigma(X.\theta))$



- **Explications :**

Si notre modèle prédit $\sigma(x) = 1$ alors que $y = 0$, on doit pénaliser la machine par une **grande** erreur (un grand coût). Cette fois $-\log(1-0)$ donne la même courbe, inversée sur l'axe vertical.

Régression Logistique

Fonction Coût complète

- Pour écrire la Fonction Coût en une seule équation, on utilise l'astuce de séparer les cas $y = 0$ et $y = 1$ avec une annulation :

$$J(\theta) = \frac{-1}{m} \sum \mathbf{y} \times \log(\sigma(X.\theta)) + (\mathbf{1 - y}) \times \log(1 - \sigma(X.\theta))$$

- Dans le cas où $y = 0$, il nous reste :

$$J(\theta) = \frac{-1}{m} \sum \cancel{0 \times \log(\sigma(X.\theta))} + \mathbf{1 \times \log(1 - \sigma(X.\theta))}$$

- Et dans le cas où $y = 1$

$$J(\theta) = \frac{-1}{m} \sum \mathbf{1 \times \log(\sigma(X.\theta))} + \cancel{0 \times \log(1 - \sigma(X.\theta))}$$

Régression Logistique

Rappel Loi de Bernoulli

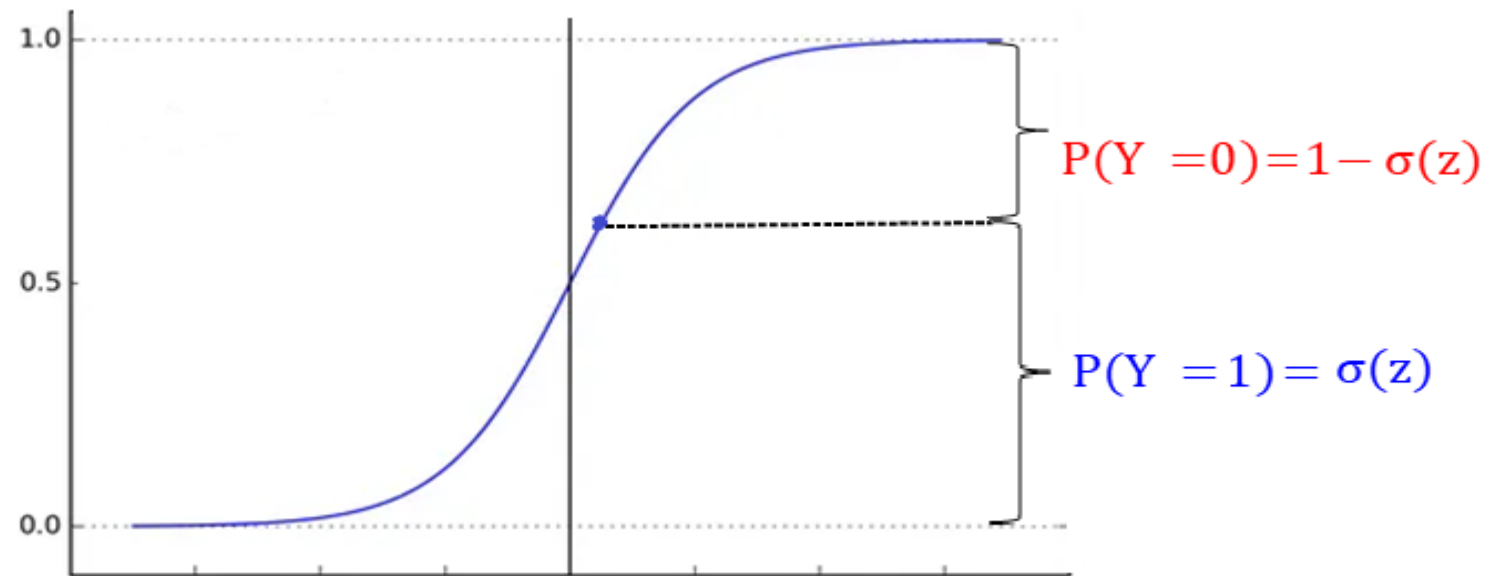
- Ces probabilités suivent en fait une loi de Bernoulli : $P(Y = y) = \sigma(z)^y \times (1 - \sigma(z))^{1-y}$

- Sachant que :

$$P(Y = 0) = \sigma(z)^0 \times (1 - \sigma(z))^{1-0} = (1 - \sigma(z))$$

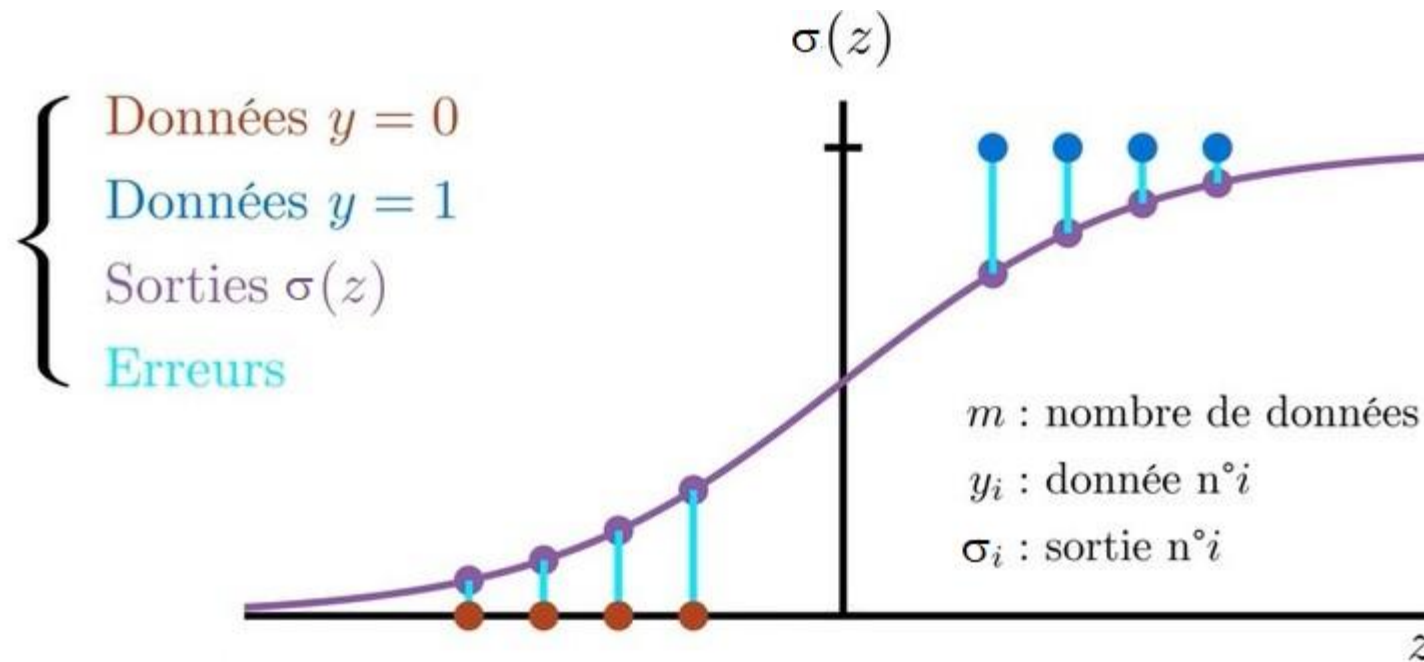
$$P(Y = 1) = \sigma(z)^1 \times (1 - \sigma(z))^{1-1} = \sigma(z)$$

- (Rappel : $x^0 = 1$)



Régression Logistique

- A présent, notre objectif consiste à optimiser les paramètres a , et b afin d'obtenir le modèle le plus performant possible, c'est-à-dire celui qui minimise au maximum les divergences entre les sorties $\sigma(z)$ et les données réelles y . Cette optimisation s'appuie sur l'utilisation d'une **fonction de coût**, également appelée "**Loss Function**" en anglais, qui sert à évaluer de manière quantitative les erreurs commises par le modèle.



Régression Logistique

- Une façon d'évaluer la performance de notre modèle c'est de calculer sa **vraisemblance (Likelihood en anglais)** en statistiques. Pour ce faire nous allons calculer simplement le produit de toutes ces probabilités en utilisant la loi de Bernoulli.

$$L = \prod_{i=1}^m P(Y = y_i) = \prod_{i=1}^m \sigma_i^{y_i} \times (1 - \sigma_i)^{1-y_i}$$

- Si le résultat de cette évaluation approche à 100%, cela indiquera que le modèle est vraisemblable, tandis qu'une proximité avec 0% traduira une forte improbabilité de notre modèle.

Régression Logistique

- Comme en calcul un produit de probabilité, des valeurs entre 0 et 1, on se rend compte que on tombe sur un résultat proche de zéro. Plus on a de nombreux plus le résultat tend vers zéro.
- La solution est d'utiliser le **Logarithme** car le logarithme d'un produit nous donne la somme des logarithmes.

$$\log(ab) = \log(a) + \log(b)$$

- on aura donc :

$$Log(L) = Log\left(\prod_{i=1}^m \sigma_i^{y_i} \times (1 - \sigma_i)^{1-y_i}\right)$$

- Or le logarithme est fonction monotone croissante alors elle conserve l'ordre de nos termes. cela signifie que lorsqu'on cherchera le maximum de notre vraisemblance il nous suffira de chercher le maximum du **log** de la **vraisemblance**.

$$Argmax(Likelihood) = Argmax(\log(Likelihood))$$

Régression Logistique

- Rappelons que :

$$\log(ab) = \log(a) + \log(b) \quad \text{et} \quad \log(a^y) = y \log(a)$$

$$\begin{aligned} LL &= \text{Log}\left(\prod_{i=1}^m \sigma_i^{y_i} \times (1 - \sigma_i)^{1-y_i}\right) \\ &= \sum_{i=1}^m (\log(\sigma_i^{y_i} \times (1 - \sigma_i)^{1-y_i})) \\ &= \sum_{i=1}^m (\log(\sigma_i^{y_i}) + \text{Log}((1 - \sigma_i)^{1-y_i})) \end{aligned}$$

$$LL = \sum_{i=1}^m (y_i \text{Log}(\sigma_i) + (1 - y_i) \text{Log}(1 - \sigma_i))$$

Régression Logistique

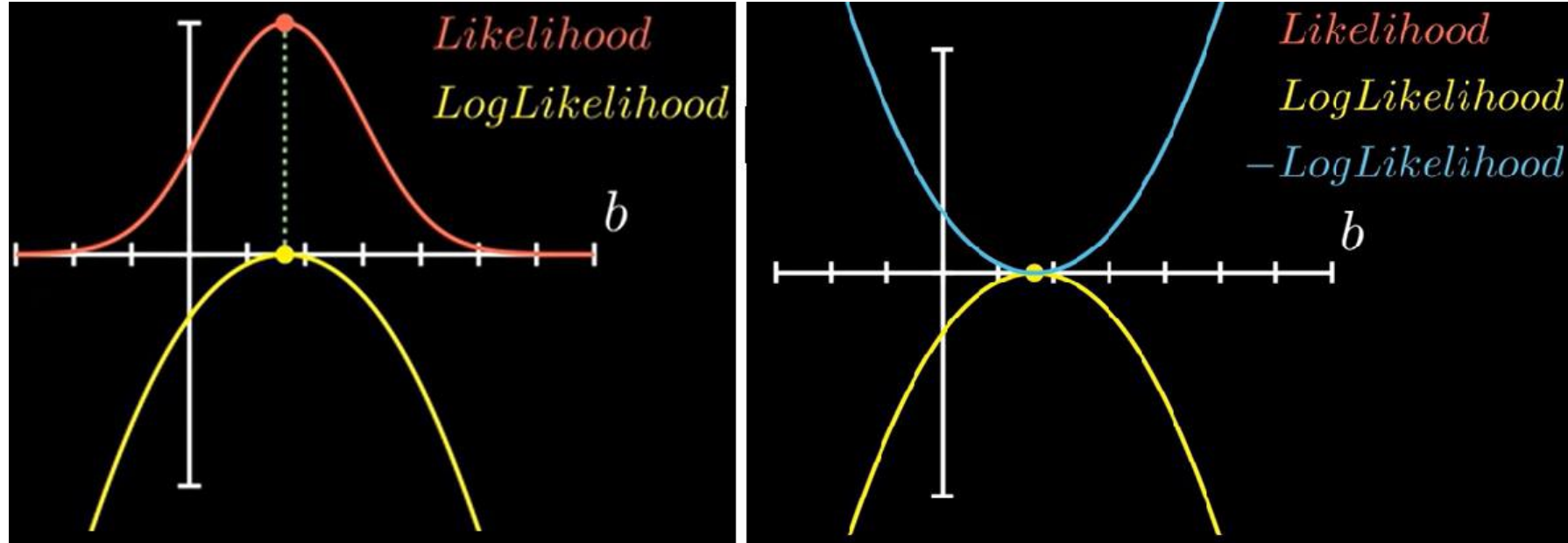
- Ce que nous avons accompli jusqu'à présent consiste à calculer le logarithme de la vraisemblance et notre objectif est de le maximiser afin d'obtenir le modèle le plus plausible.
- Cependant, en mathématiques, il n'existe pas d'algorithmes de maximisation typiques, on opte plutôt généralement pour des algorithmes de minimisation.
- Ainsi, pour maximiser une fonction $f(x)$, l'approche courante consiste à minimiser la fonction $(-f(x))$.
- Dans ce contexte, pour optimiser la vraisemblance de notre modèle, nous visons en réalité à minimiser la version négative de sa fonction.

Régression Logistique

- *Maximiser*($f(x)$) = *Minimiser*($-f(x)$) ce qui va nous donner la Fonction **Log Loss**

$$L = -\frac{1}{m} \sum_{i=1}^m (y_i \text{Log}(\sigma_i) + (1 - y_i) \text{Log}(1 - \sigma_i))$$

- le facteur $\frac{1}{m}$ c'est juste un facteur pour normaliser le résultat



Gradient Descent pour la Régression Logistique

- L'algorithme de Gradient Descent s'applique exactement de la **même manière** que pour la régression linéaire. En plus, la dérivée de la Fonction Coût est la même aussi ! On a :

- Gradient:*
$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} \sum (\sigma(X.\theta) - y).X$$

- Gradient Descent:*
$$\theta = \theta - \alpha \times \frac{\partial J(\theta)}{\partial \theta}$$

Résumé de la Régression Logistique

- Modèle:*
$$\sigma(X.\theta) = \frac{1}{1 + e^{-X.\theta}}$$

- Fonction Coût:*
$$J(\theta) = \frac{-1}{m} \sum y \times \log(\sigma(X.\theta)) + (1 - y) \times \log(1 - \sigma(X.\theta))$$

- Gradient:*
$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} X^T . (\sigma(X.\theta) - y)$$

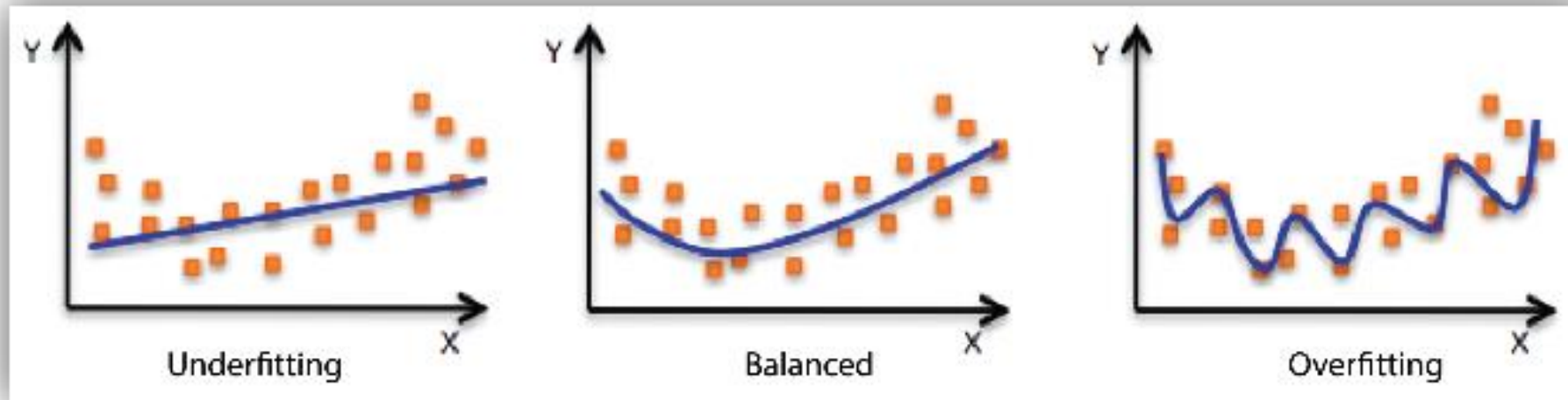
- Gradient Descent:*
$$\theta = \theta - \alpha \times \frac{\partial J(\theta)}{\partial \theta}$$

$$Y = X\theta + \varepsilon$$
$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, \quad \varepsilon = \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{pmatrix} \quad \text{et} \quad X = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}$$

Régression Logistique

Lorsqu'il s'agit de former des modèles, deux problèmes majeurs peuvent se poser : **l'overfitting** et **l'underfitting**.

- **Overfitting** : Il y a surajustement lorsque le modèle est performant sur l'ensemble d'apprentissage mais moins bien sur les données non vues (test).
- **Underfitting** : Il y a sous-adaptation lorsqu'il n'y a pas de bonnes performances sur l'ensemble de formation ni sur l'ensemble de test.



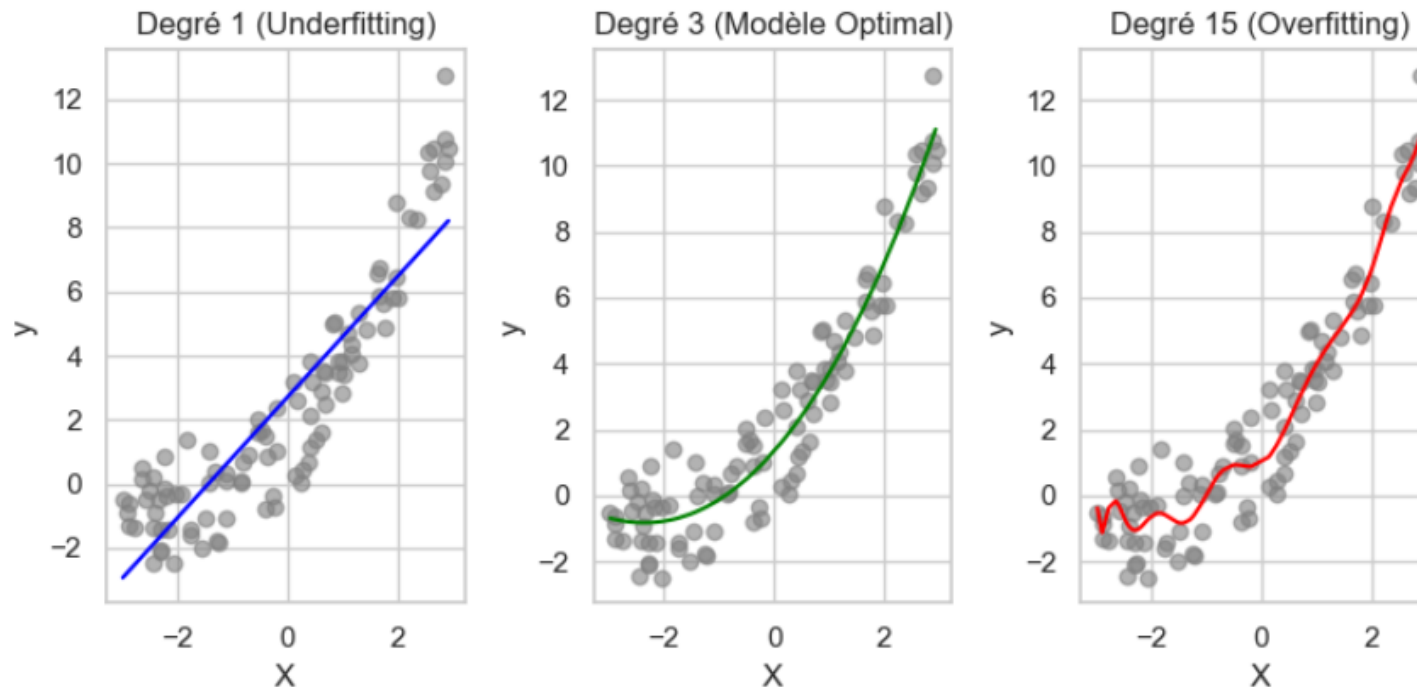
Overfitting (Surapprentissage) et Underfitting (Sous-apprentissage)

- **Surapprentissage (Overfitting) :**

- L'overfitting se produit lorsque le modèle est trop complexe et s'ajuste trop bien aux données d'entraînement, capturant non seulement la tendance sous-jacente mais aussi le bruit (erreur aléatoire). Cela peut conduire à une mauvaise généralisation sur de nouvelles données.

- **Sous-apprentissage (Underfitting) :**

- L'underfitting se produit lorsqu'un modèle trop simple ne parvient pas à capturer les relations sous-jacentes dans les données. Il a une performance faible aussi bien sur les données d'entraînement que sur les données de test



La régularisation

- La **régularisation** est une technique qui évite le surajustement (**overfitting**) en ajoutant une pénalité sur la complexité du modèle.
- Elle agit en contraignant les poids du modèle pour éviter qu'ils ne prennent des valeurs extrêmes.
- Elle aide à stabiliser les estimations et à améliorer la généralisation.
- **Les trois principales méthodes de régularisation :**
 - **Ridge Regression (L2)** : Ajoute une pénalité proportionnelle au carré des coefficients.
 - **Lasso Regression (L1)** : Ajoute une pénalité proportionnelle à la valeur absolue des coefficients.
 - **Elastic Net** : Combinaison des deux approches (L1 + L2)

La régularisation

- **Formule Générale de la Fonction Coût Régularisée**

$$J(\theta) = J_{\text{original}}(\theta) + \lambda R(\theta)$$

où :

- $J_{\text{original}}(\theta)$ est la fonction coût classique (MSE, log-loss, etc.).
- $R(\theta)$ est le terme de régularisation.
- λ est l'hyperparamètre de régularisation qui contrôle l'importance de la pénalité.

Régression Ridge (L2)

- La **Régression Ridge** ajoute une pénalité L2, qui contraint la somme des carrés des coefficients.
- **Formule du coût Ridge :**

$$J(\theta) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n \theta_j^2$$

où :

- λ est le paramètre de régularisation.
- $\sum_{j=1}^n \theta_j^2$ est la pénalité L2.

Effet : Réduit la variance du modèle en réduisant la magnitude des coefficient

Régression Lasso (L1)

- La **Régression Lasso** ajoute une pénalité L1, qui force certains coefficients à être exactement **nuls**, ce qui favorise la **sélection de variables**.

- **Formule du coût Lasso :**

$$J(\theta) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n |\theta_j|$$

Effet : Supprime les coefficients les moins importants et **simplifie le modèle**.

Elastic Net - Hybride de L1 et L2

- L'**Elastic Net** combine **L1 (Lasso)** et **L2 (Ridge)**, ce qui offre un compromis entre sélection de variables et stabilisation des coefficients.
- **Formule du coût Elastic Net**

$$J(\theta) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda_1 \sum_{j=1}^n |\theta_j| + \lambda_2 \sum_{j=1}^n \theta_j^2$$

Effet : Garde les avantages du Lasso (suppression de variables) et du Ridge (réduction de la variance).

La régularisation

- **Objectif** : Réduire la complexité du modèle en pénalisant la somme des valeurs absolues des coefficients (L_1 norm), ce qui peut forcer certains coefficients à devenir exactement zéro, effectuant ainsi une sélection de variables.
- **Fonction de coût modifiée** : La régularisation Lasso ajoute une pénalité basée sur la somme des valeurs absolues des coefficients à la fonction de coût de l'erreur quadratique moyenne.

Formule :

Si $\hat{y} = Xw$ est la prédiction, la fonction de coût à minimiser devient :

$$J(w) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum |w_j|$$

- y_i : valeurs réelles,
- $\hat{y}_i$: prédictions,
- w_j : coefficients du modèle,
- λ : paramètre de régularisation (contrôle l'importance de la pénalité).

Effet :

- $\lambda = 0$: Pas de régularisation, le modèle est une régression linéaire classique.
- $\lambda > 0$: Les coefficients sont réduits, certains pouvant devenir exactement zéro, ce qui conduit à une sélection automatique des variables les plus importantes.

La régularisation

Avantages :

- **Sélection de variables** : Lasso peut automatiquement ignorer les variables non pertinentes en leur attribuant des coefficients nuls.
- **Simplicité** : Permet d'obtenir des modèles plus simples et interprétables.

Inconvénients :

- Peut ne pas fonctionner correctement si les variables sont fortement corrélées, car il choisit arbitrairement une variable parmi les corrélées pour attribuer un coefficient non nul.

Comparaison avec Ridge :

- **Lasso (L1)** tend à produire des modèles plus simples avec moins de variables (sparse models).
- **Ridge (L2)** réduit les coefficients sans les annuler totalement, ce qui est utile quand toutes les variables ont une certaine importance.

La régularisation

Régularisation Lasso

- Exemple 2 : **Régularisation ℓ_1**
- Pour un modèle paramétrique de paramètres $\theta_1, \theta_2, \dots, \theta_d$,
 $\Omega(\theta) = \left\| \theta \right\|_1 = \sum_{j=1}^d |\theta_j|$ contrôle **le nombre de paramètres non nuls** → **parcimonie** (sparsity)
- **Lasso** (Least Absolute Sparse Selection Operator) :

$$\arg \min_{\beta \in \mathbb{R}^{p+1}} \frac{1}{n} \left(y - X\beta \right)^\top \left(y - X\beta \right) + \lambda \left\| \beta \right\|_1$$

- Résolu par l'algorithme du gradient.

La régularisation

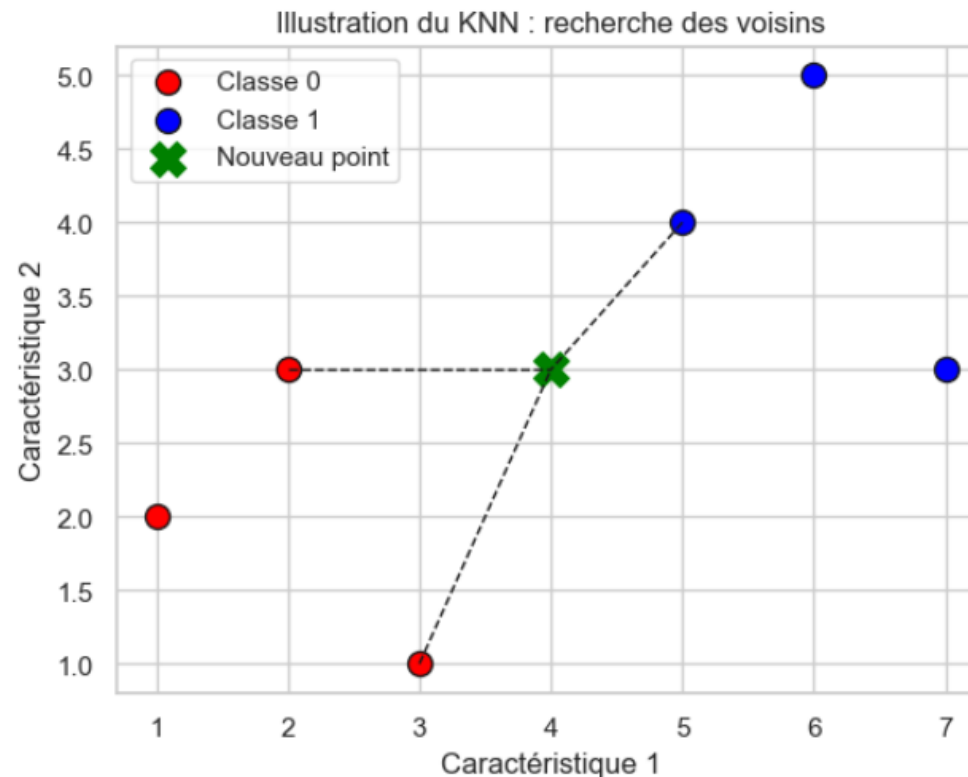
Régularisation Lasso

- La régression qualifiée de "Lasso" consiste à minimiser la somme des valeurs absolues des coefficients, le tout pondéré par un facteur (noté λ).
- On se retrouve avec à minimiser l'expression suivante:
 - Somme des résidus quadratiques (RSS) + $\lambda * (\text{somme des valeurs absolues des coefficients} - \|\cdot\|_1)$
- Les propriétés sont assez similaires entre la régression Ridge et Lasso, les différences fondamentales consistent en:
- Lorsque λ tend vers l'infini ($+\infty$), les coefficients vont tendre vers 00 (mais sans jamais l'atteindre) dans le cadre de "Ridge" mais pourront atteindre 0 pour "Lasso".
- "Ridge" est davantage adapté dans le cadre où les variables indépendantes sont fortement corrélées. Alors que "Lasso" permet de "sélectionner" les covariables en vue de rendre le modèle plus simple (vu que les coefficients peuvent devenir nuls).
- Vu que la norme L_1 est employée, cela a des répercussions sur les techniques d'optimisations possibles en vue de minimiser ce système.

KNN (K-Nearest Neighbors)

KNN (K-Nearest Neighbors)

- Le KNN est un **modèle d'apprentissage supervisé** utilisé pour la classification et la régression
- Il est **non paramétrique**, ce qui signifie qu'il ne fait pas d'hypothèses sur la distribution des données.
- **Principe** : Un point est classé selon les **K** voisins les plus proches dans l'espace des caractéristiques.

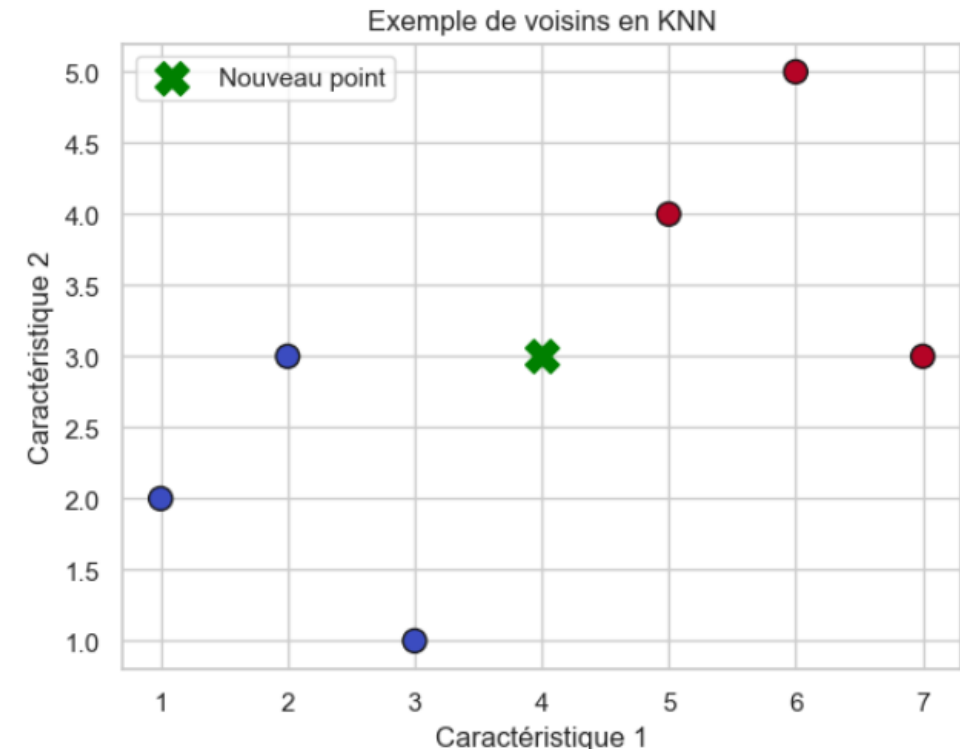


KNN (K-Nearest Neighbors)

Principe de base du KNN

- On choisit un nombre de voisins K.
- On calcule la distance entre le point à prédire et tous les autres points du dataset.
- On sélectionne les **K** plus proches voisins.
- On effectue un **vote majoritaire** pour classier le point.
- Équation de la distance euclidienne :

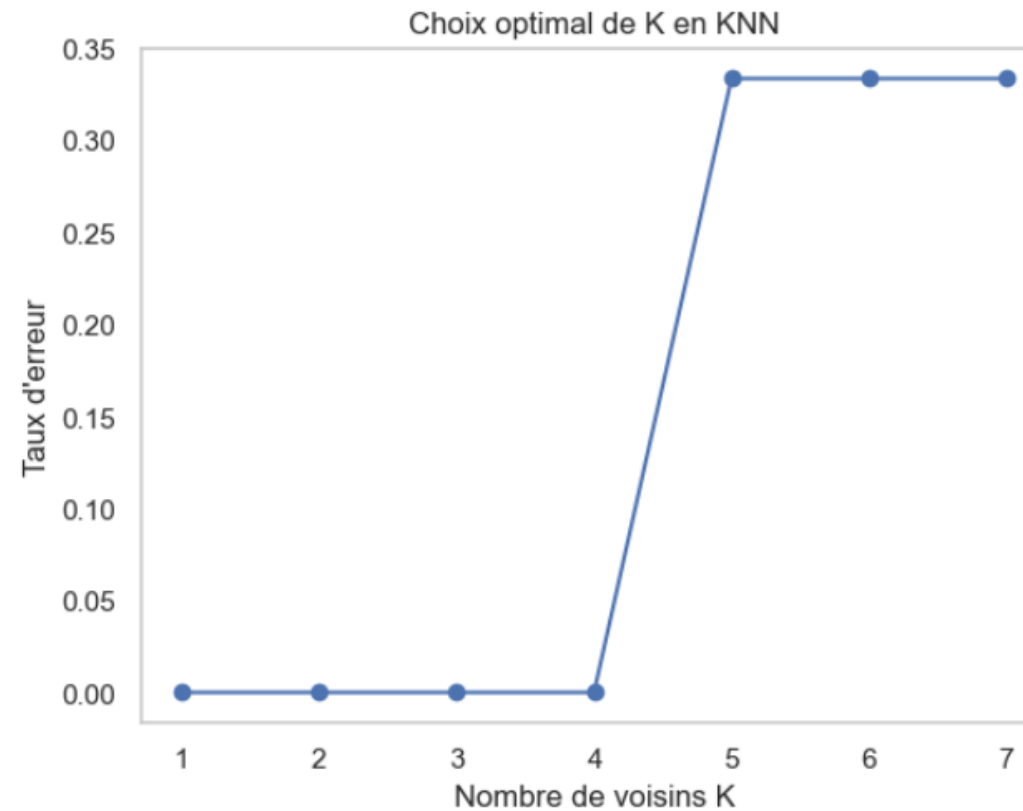
$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2}$$



KNN (K-Nearest Neighbors)

Choisir le bon nombre de voisins K

- Si **K** est trop petit → modèle **sensible au bruit** et overfitting
- Si **K** est trop grand → modèle **moins flexible**
- On choisit **K** en testant plusieurs valeurs et en utilisant la **validation croisée**



KNN (K-Nearest Neighbors)

La validation croisée

- **Définition :** La **validation croisée** est une technique utilisée pour **évaluer les performances** d'un modèle en le testant sur plusieurs sous-ensembles de données. Elle permet d'éviter le **sur-apprentissage** (overfitting) et d'obtenir une estimation plus fiable de l'erreur du modèle.
- **Fonctionnement (k-fold Cross-Validation)**
 - Diviser les données en k sous-ensembles (folds).
 - Entraîner le modèle sur $k-1$ sous-ensembles et tester sur le dernier.
 - Répéter l'opération k fois, en changeant le sous-ensemble de test.
 - Calculer la moyenne des scores obtenus sur les k itérations.

KNN (K-Nearest Neighbors)

Distance utilisée en KNN

- Différentes distances sont utilisées pour mesurer la similarité entre points :

- **Distance Euclidienne** (cas classique)

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2}$$

- **Distance de Manhattan**

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sum_{k=1}^n |x_{ik} - x_{jk}|$$

- **Distance de Minkowski** (généralisation)

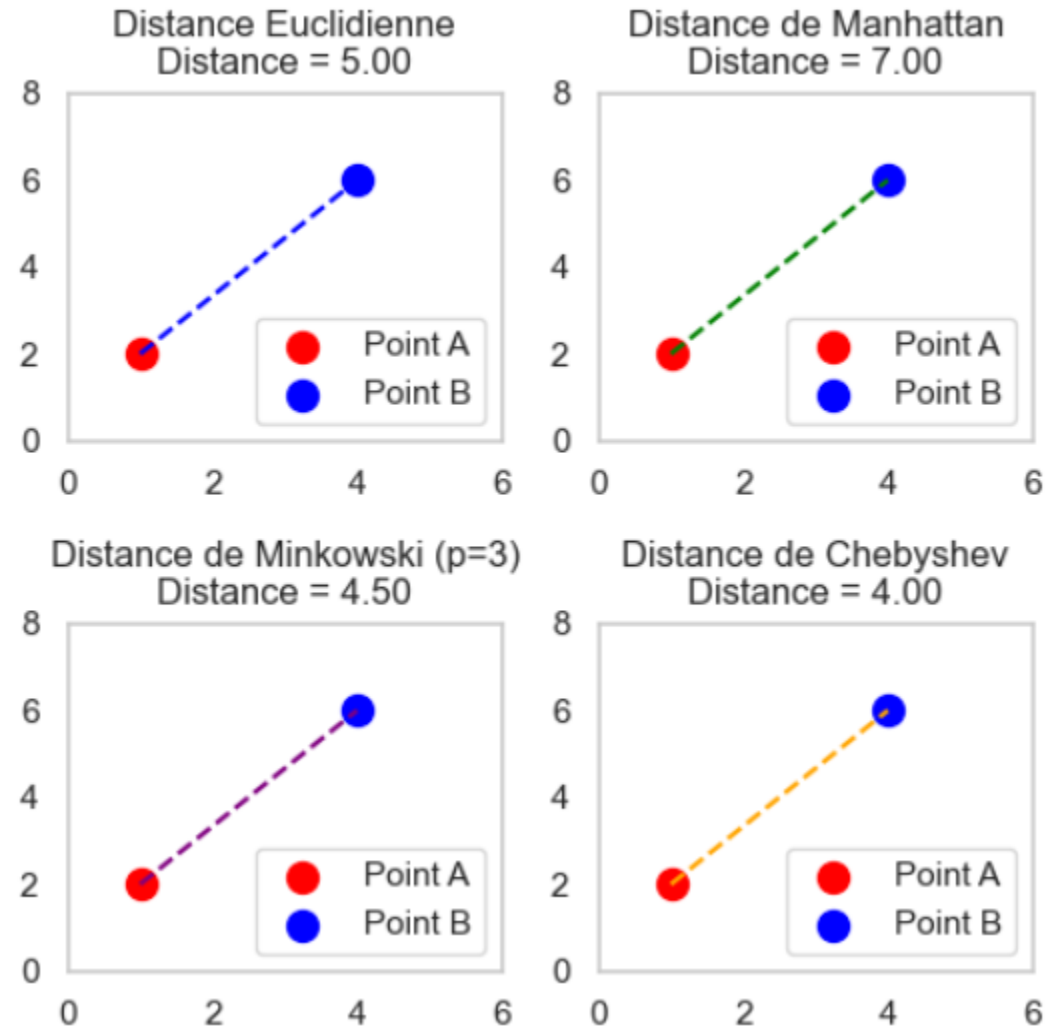
$$d(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{k=1}^n |x_{ik} - x_{jk}|^p \right)^{\frac{1}{p}}$$

- **Distance de Chebyshev**

$$D_{\text{Chebyshev}}(A, B) = \max_i |x_i - y_i|$$

KNN (K-Nearest Neighbors)

Distance utilisée en KNN



KNN (K-Nearest Neighbors)

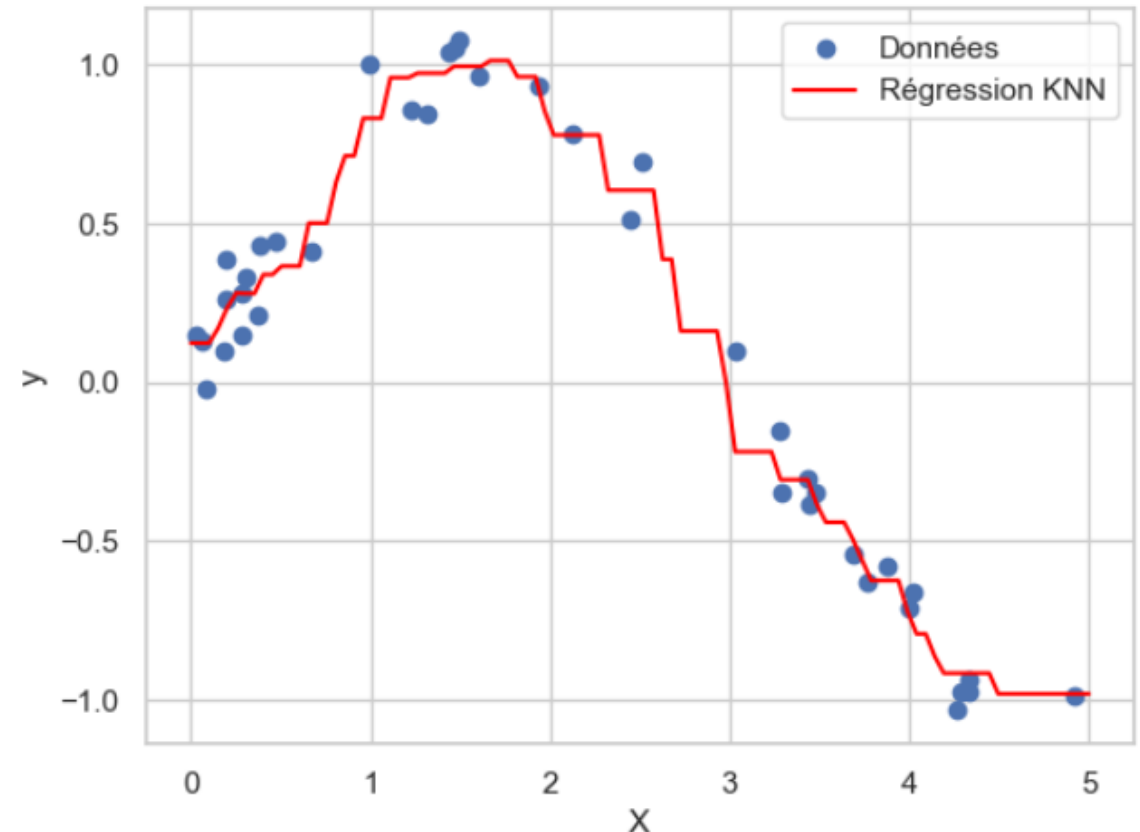
KNN pour la régression

- Contrairement à KNN pour la classification, KNN pour la régression prédit une valeur continue plutôt qu'une catégorie.
- Il utilise la moyenne (ou parfois la médiane) des K voisins les plus proches.

$$\hat{y} = \frac{1}{K} \sum_{i=1}^K y_i$$

où :

- $\hat{y}$ est la valeur prédite,
- K est le nombre de voisins,
- y_i sont les valeurs cibles des K voisins les plus proches.



KNN (K-Nearest Neighbors)

Forces et Limites de KNN Régression

- **Avantages :**

- Facile à comprendre et à implémenter
- Aucune hypothèse forte sur la distribution des données
- Peut bien fonctionner pour des relations complexes non linéaires

- **Inconvénients :**

- Sensible au choix de K
- Coût de calcul élevé pour de grandes bases de données
- Performances faibles si les dimensions sont élevées (problème de la malédiction de la dimension)

KNN (K-Nearest Neighbors)

Exemple : Reconnaissance d'image avec KNN

- KNN peut être utilisé pour la reconnaissance d'images, où l'objectif est de classer des images en fonction de leur contenu (ex. reconnaître des chiffres, des objets, des lettres, etc.).
- Chaque image est traitée comme un vecteur de caractéristiques (features) dans un espace de dimensionnalité plus élevée (souvent obtenues par des techniques comme l'extraction de caractéristiques).
- **Exemple concret :**
 - Utilisation de KNN pour la classification des **chiffres manuscrits** dans le jeu de données **MNIST**.
- **Processus :**
 1. Convertir l'image en un vecteur de caractéristiques (par exemple, un tableau de pixels aplatis).
 2. Pour une nouvelle image, **calculer les distances** entre cette image et toutes les autres images de l'ensemble d'entraînement.
 3. **Classer** l'image en fonction des étiquettes des K voisins les plus proches.

KNN (K-Nearest Neighbors)

Exemple : Reconnaissance d'image avec KNN

- Etapes de l'implémentation :
 - Chargement des données MNIST (disponibles dans **sklearn.datasets** ou **tensorflow.keras.datasets**).
 - Aplatir les images en vecteurs.
 - Appliquer l'algorithme KNN pour classifier les images en chiffres (0-9).
 - Calculer la précision du modèle.

SVM : Support Vector Machines

Support Vector Machines

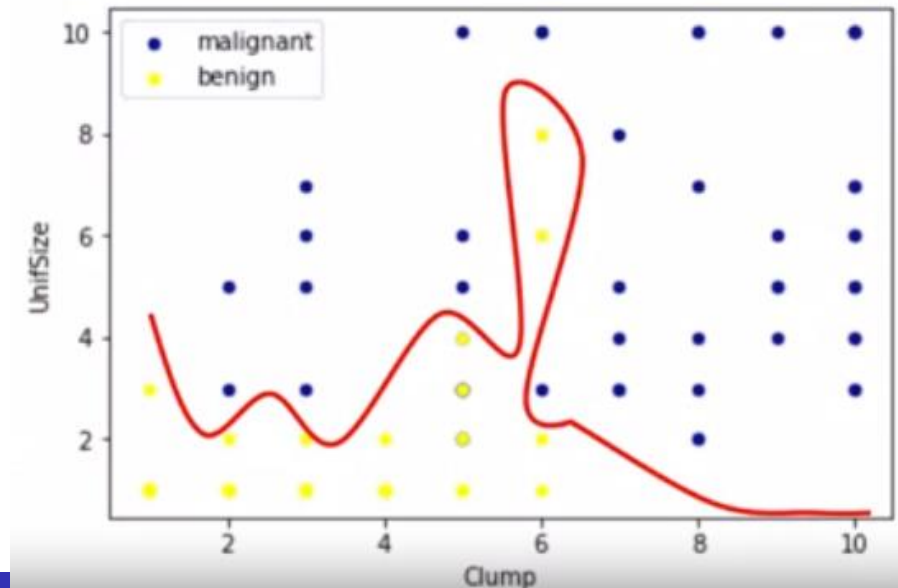
- **Qu'est-ce qu'un SVM ?**

- Un SVM (Support Vector Machine) est un algorithme d'apprentissage supervisé utilisé pour la classification et la régression.
- Il vise à trouver un hyperplan optimal qui sépare les classes de manière optimale.
- L'objectif est de maximiser la marge entre les classes tout en minimisant les erreurs de classification.

- **Concept de Marge**

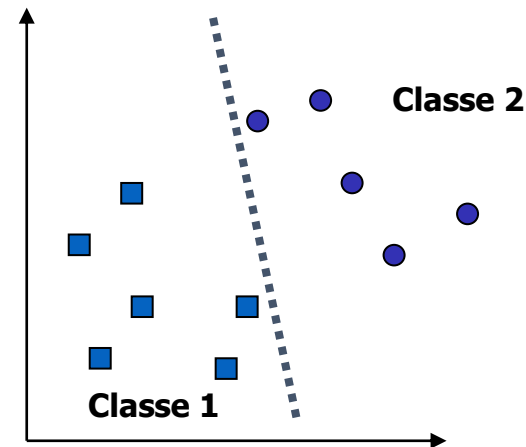
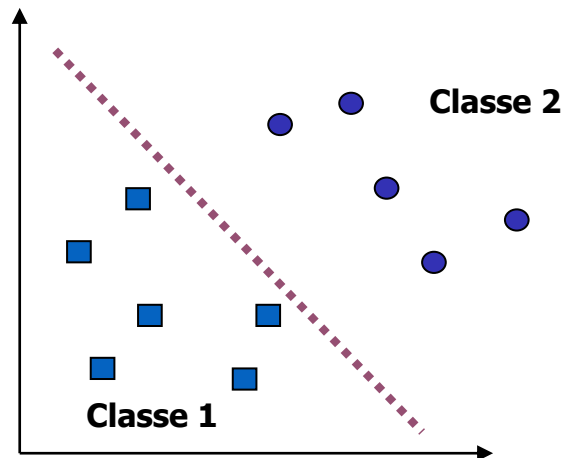
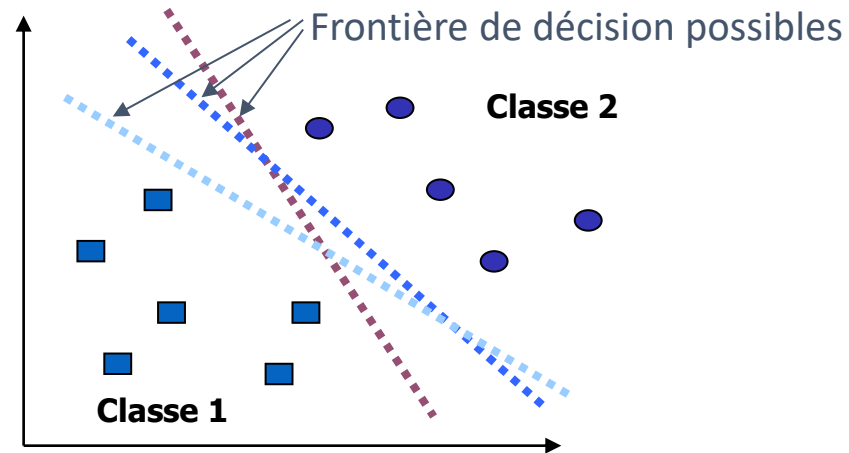
- La marge est la distance entre l'hyperplan de décision et les points les plus proches de chaque classe (vecteurs supports).
- Un SVM optimal maximise cette marge pour assurer une meilleure généralisation sur des nouvelles données.

Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
5	1	1	1	2	1	3	1	1	benign
5	4	4	5	7	10	3	2	1	benign
3	1	1	1	2	2	3	1	1	malignant
6	8	8	1	3	4	3	7	1	benign
4	1	1	3	2	1	3	1	1	benign
8	10	10	8	7	10		7	1	malignant
1	1	1	1	2	10	3	1	1	benign
2	1	2	H	2	1	3	1	1	benign
2	1	1	1	2	1	1	1	5	benign
4	2	1	1	2	1	2	1	1	benign



Support Vector Machines

- Plusieurs surfaces de décision existent pour séparer les classes ;
- laquelle choisir ?



- Pour minimiser la sensibilité au bruit,
- la droite de décision doit être aussi éloignée que possible des données de chaque classe

Support Vector Machines

- Soit un ensemble de données d'entraînement

$$\{(x_i, y_i) | x_i \in \mathbb{R}^n, y_i \in \{-1, 1\}\}$$

- L'hyperplan est défini par :

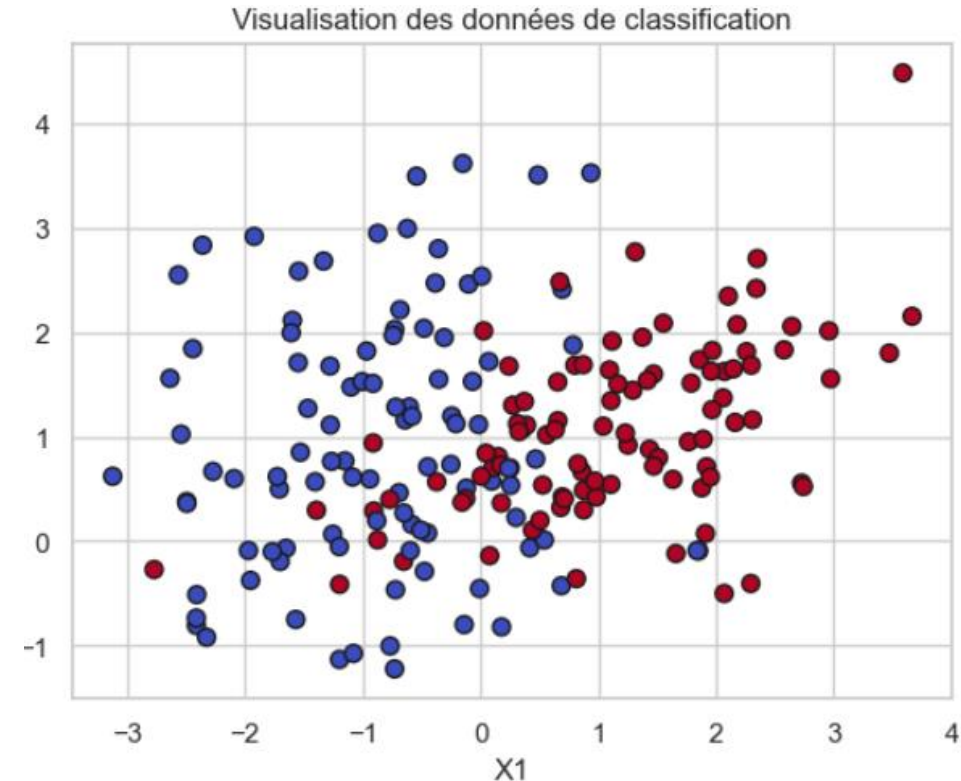
$$w^T x + b = 0$$

- L'objectif est de maximiser la marge :

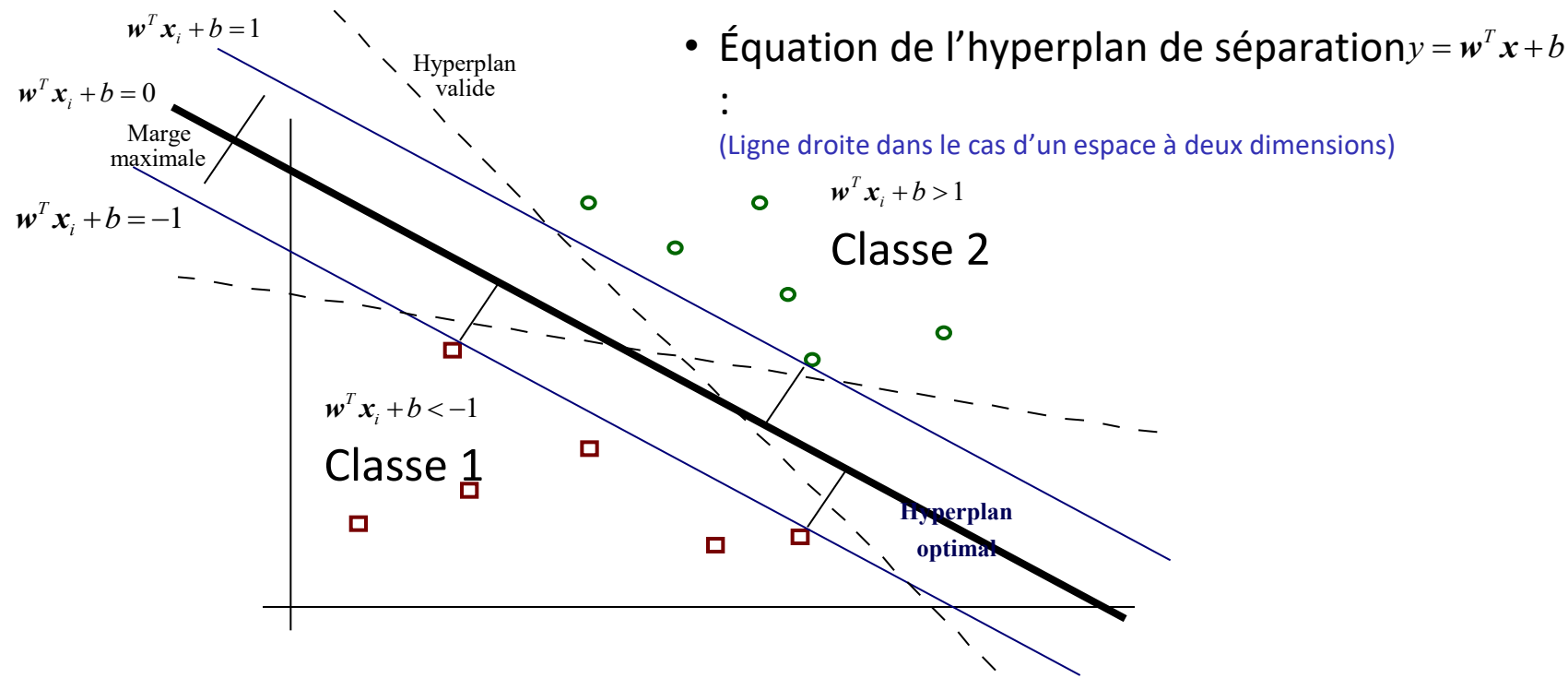
$$\frac{2}{\|w\|}$$

- Tout en respectant la contrainte :

$$y_i(w^T x_i + b) \geq 1$$



Support Vector Machines



- L'hyperplan est représenté par la ligne centrale noire.
- Les marges sont les lignes bleu à -1 et +1.
- Les points les plus proches de l'hyperplan sont les vecteurs supports

Support Vector Machines

Optimisation de la marge

- Distance normale d'un point à l'hyperplan :

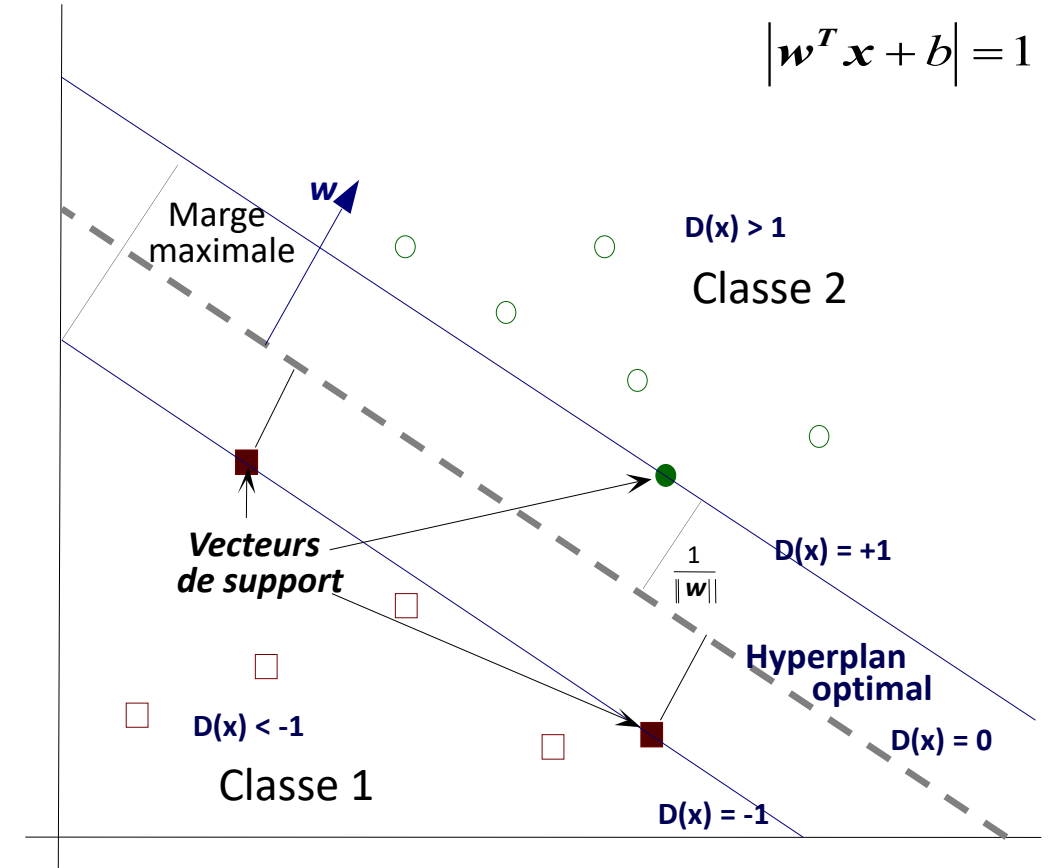
$$D(\mathbf{x}) = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$$

- Marge max. avant d'atteindre les frontières des deux classes ($|\mathbf{w}^T \mathbf{x} + b| = 1$) :

$$m = \frac{2}{\|\mathbf{w}\|}$$

- Maximiser m revient à minimiser $\|\mathbf{w}\|$ tout en préservant le pouvoir de classification :

$$\min \frac{1}{2} \|\mathbf{w}\|^2 \text{ sous la contrainte : } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \forall i$$



Support Vector Machines

- Pour Résoudre le problème $\min \frac{1}{2} \|w\|^2$ sous la contrainte : $y_i(w^T x_i + b) \geq 1, \forall i$, On introduit les multiplicateurs de Lagrange $\alpha_i \geq 0$ pour transformer le problème en une **forme duale** :

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i [y_i(w \cdot x_i + b) - 1]$$

- En annulant les dérivées partielles de L par rapport à w et b :

- $w = \sum_i \alpha_i y_i x_i$
- $\sum_i \alpha_i y_i = 0$

- On remplace w dans L pour obtenir la **forme duale** :
$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i \cdot x_j$$

sous les contraintes :

- $\alpha_i \geq 0$
- $\sum_i \alpha_i y_i = 0$

w et b sont alors calculés à partir des α_i .

Support Vector Machines

Le paramètre C et la régularisation

- C est un hyperparamètre qui contrôle le compromis entre une marge large et une faible erreur d'entraînement.
- L'objectif du SVM est de trouver un **hyperplan** qui sépare au mieux les classes avec une marge optimale.
L'optimisation du SVM est donnée par :

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i$$

- C élevé : Moins de marge, moins d'erreurs d'entraînement.
- C faible : Plus de marge, mais plus d'erreurs tolérées pour éviter le surapprentissage.

Support Vector Machines

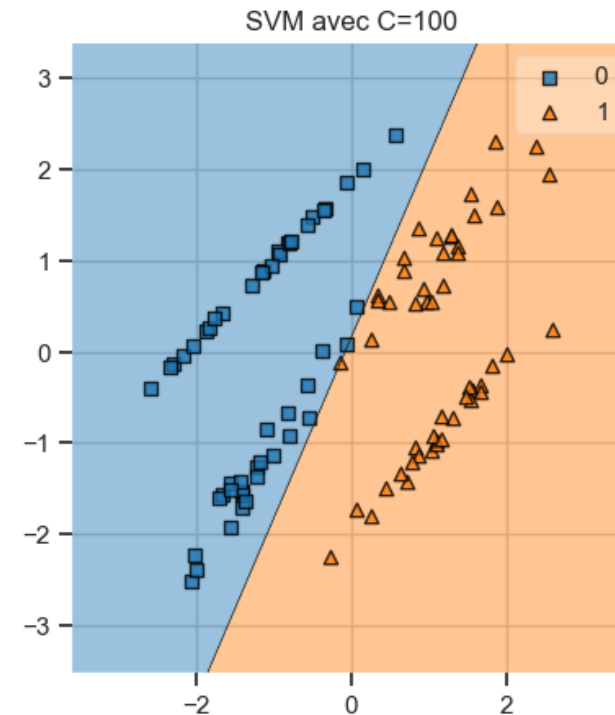
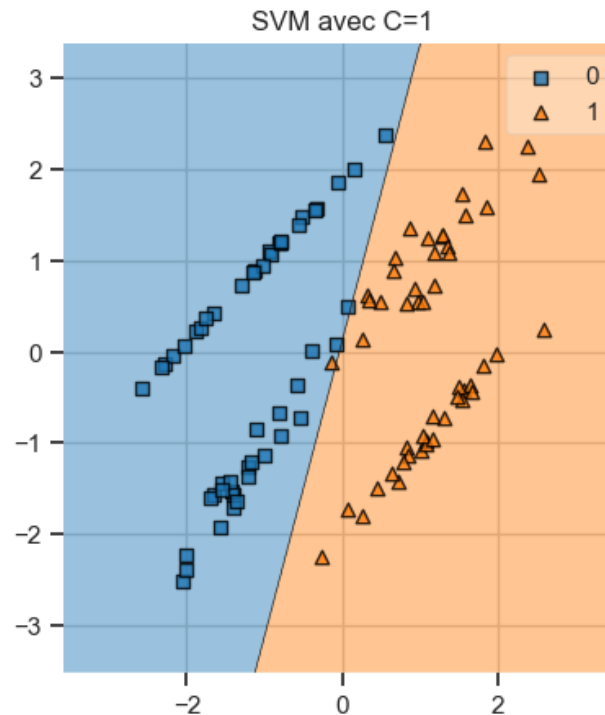
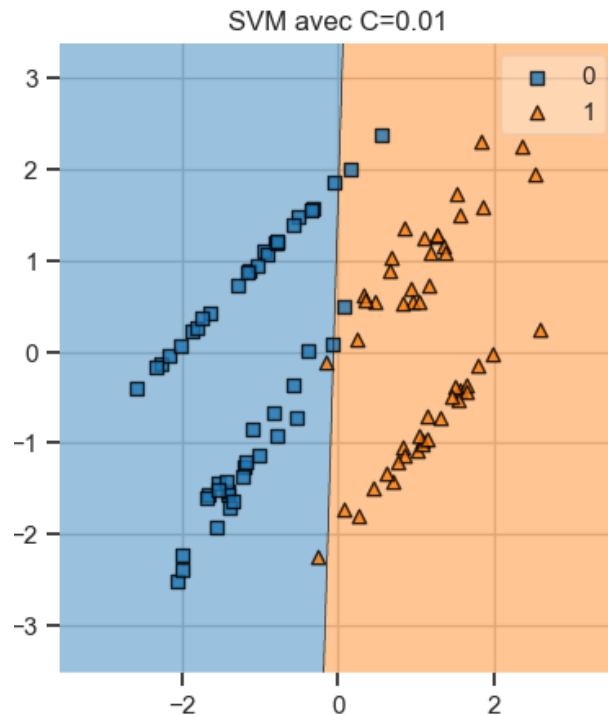
Le paramètre C et la régularisation

- **Influence d'un CCC élevé**
- Un C **élevé** signifie que l'on **pénalise fortement** les erreurs de classification.
- Cela force le modèle à classer **correctement** le plus grand nombre de points, quitte à avoir une **marge plus petite**.
- L'hyperplan sera **rigide** et risque de **mal généraliser** sur des nouvelles données (sur-apprentissage).
- **Illustration mathématique :**
 - Lorsque C est grand, la somme des erreurs $\sum \xi_i$ est minimisée, donc **les erreurs sont quasi inexistantes**. Mais cela force le modèle à trouver un hyperplan avec une **marge très fine** qui épouse trop bien les données d'entraînement.
- **Influence d'un CCC faible**
- Un C **faible** permet au modèle de **tolérer certaines erreurs**.
- Il favorise **une marge plus large**, ce qui améliore la **capacité de généralisation**.
- Cela permet au modèle d'être plus **robuste aux variations des données**.
- **Illustration mathématique :**
 - Avec un petit C, le terme $C \sum \xi_i$ a moins d'impact, donc **plus de points** sont mal classés mais **la marge est plus grande**. Cela évite de trop s'adapter aux **bruits des données**.

Support Vector Machines

Expérience visuelle : Influence de CCC sur la frontière de décision

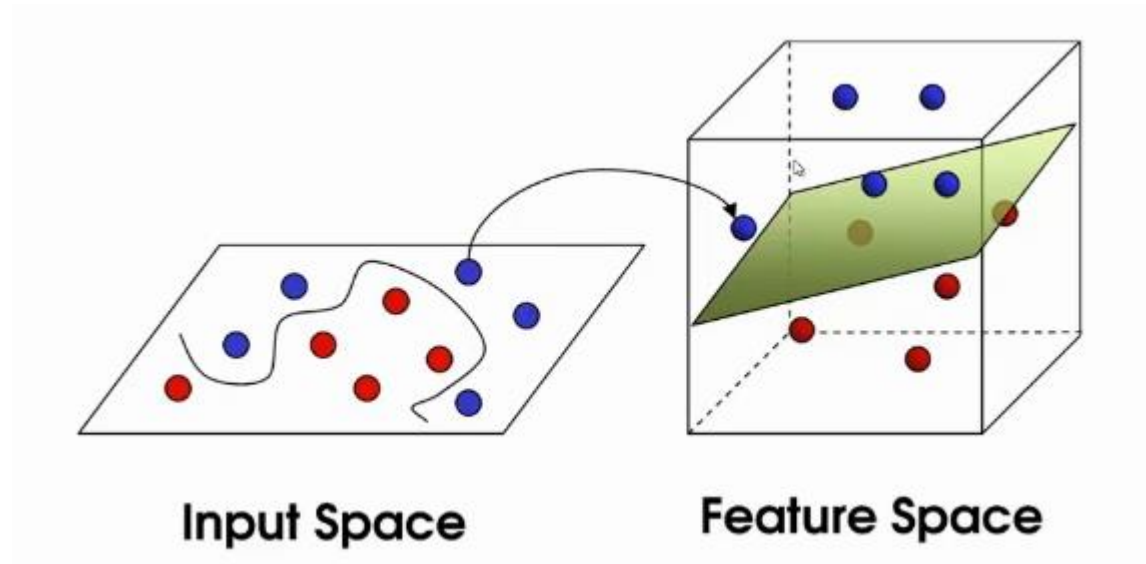
- $C=0.01$: tolère beaucoup d'erreurs, large marge.
- $C=1$: équilibre entre marge et erreur.
- $C=100$: peu d'erreurs mais marge étroite (risque de sur-apprentissage).



Support Vector Machines

SVM Non Linéaire et le Trick Noyau

- Certaines données ne sont pas séparables linéairement.
- Le **trick du noyau** permet de transformer les données en un espace de dimension supérieure où elles deviennent séparables



Support Vector Machines

SVM Non Linéaire et le Trick Noyau

- Un **noyau** est une fonction qui prend deux points d'entrée x et x' et retourne un **produit scalaire implicite** dans un espace de transformation de dimension plus élevée.
- Un noyau $K(x, x')$ remplace donc le produit scalaire standard $x \cdot x'$ dans l'algorithme SVM :

$$K(x, x') = \phi(x) \cdot \phi(x')$$

- où $\phi(x)$ est une fonction qui transforme les données en un espace de dimension supérieure. **Le noyau nous évite donc de calculer explicitement $\phi(x)$.**

Support Vector Machines

Les différents types de noyaux

- **Noyau Linéaire**

$$K(x, x') = x \cdot x'$$

- Utile quand les données sont déjà séparables linéairement.

- **Noyau Polynomial**

$$K(x, x') = (x \cdot x' + c)^d$$

- Capturer des relations non linéaires.

- **3. Noyau RBF (Gaussien)**

$$K(x, x') = e^{-\gamma \|x - x'\|^2}$$

- Idéal pour des frontières complexes.

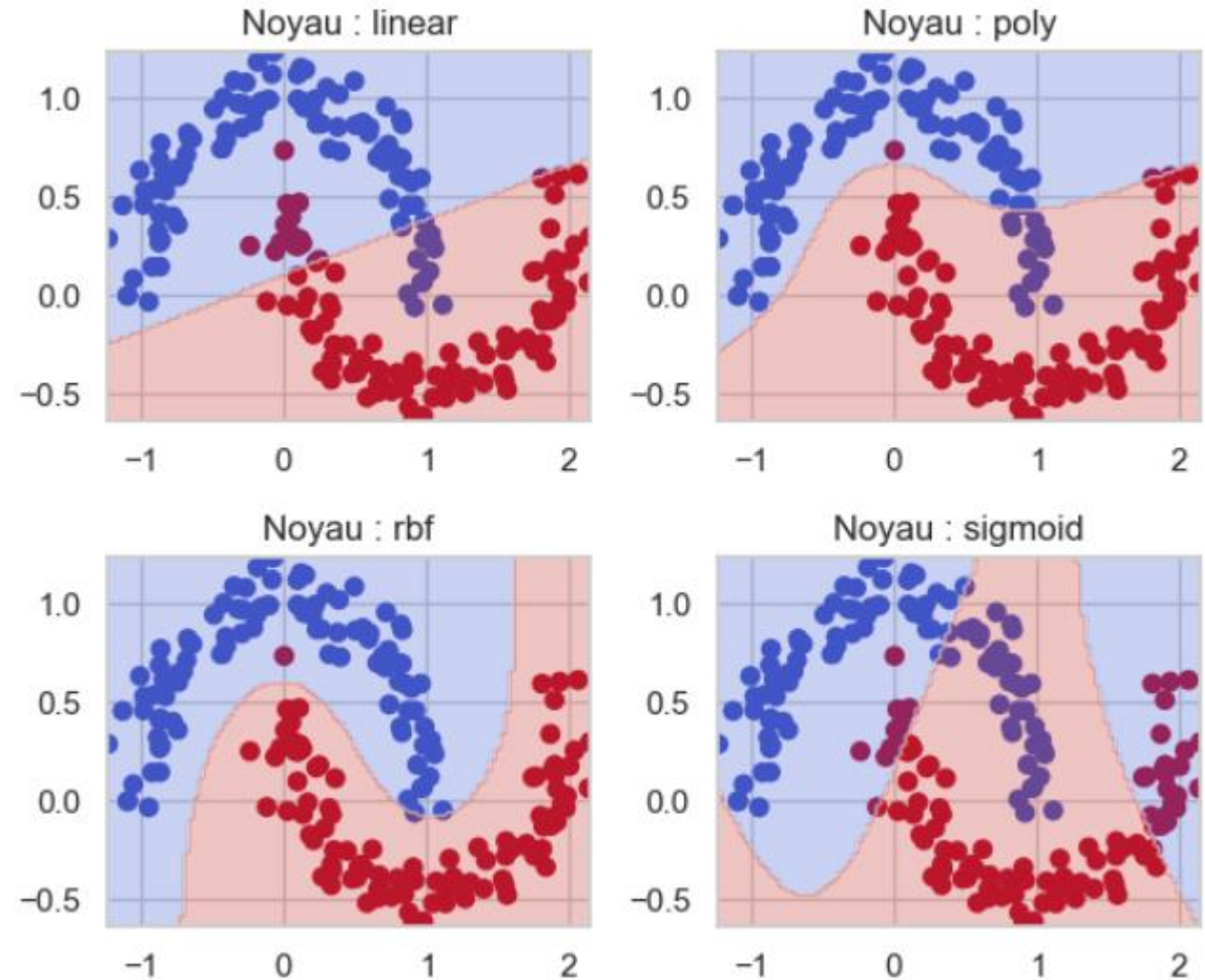
- **4. Noyau Sigmoid**

$$K(x, x') = \tanh(\alpha x \cdot x' + c)$$

Support Vector Machines

Comment choisir le bon noyau ?

- Si les données sont linéaires →
 - Utiliser un noyau linéaire.
- Si les données ont des courbes simples →
 - Utiliser un noyau polynomial (degré faible).
- Si les frontières de séparation sont complexes →
 - Utiliser un noyau RBF.
- Si les données ont une structure neuronale →
 - Essayer le noyau sigmoïde.



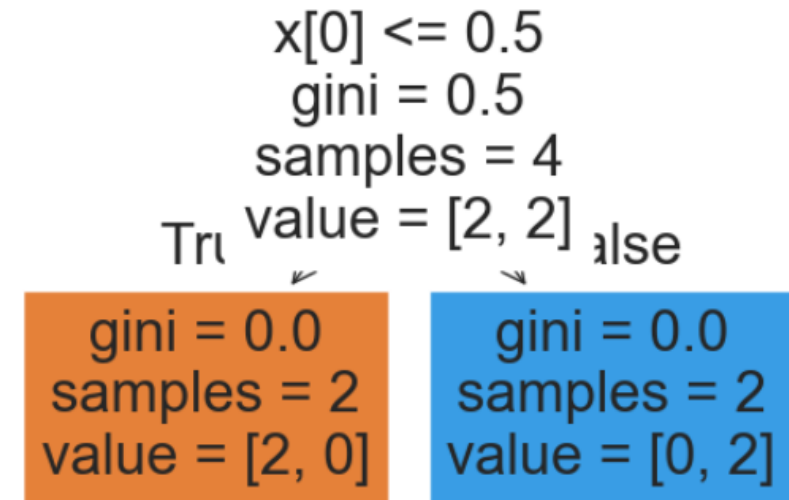
Arbres de Décision

Arbres de Décision

- Un modèle d'apprentissage supervisé utilisé pour la classification et la régression.
- Fonctionne en divisant les données en sous-ensembles basés sur des décisions conditionnelles.
- Se représente sous forme d'un graphe arborescent où chaque nœud interne correspond à une condition sur une variable et chaque feuille représente une classe ou une valeur prédite.

Structure d'un Arbre de Décision

- **Racine** : Premier nœud qui représente la première décision.
- **Nœuds internes** : Points de décision basés sur une variable.
- **Branches** : Connexions entre les nœuds.
- **Feuilles** : Résultats finaux (classe ou valeur prédite).



Arbres de Décision

- **Fonctionnement d'un Arbre de Décision**

1. Choix de la variable de séparation : Sélectionne la meilleure variable pour diviser les données.
2. Division récursive : Répète le processus pour chaque sous-ensemble.
3. Critère d'arrêt : S'arrête quand une condition est remplie (ex: profondeur max atteinte).

- **Mesure de l'impureté – Entropie**

- L'entropie mesure le désordre d'un ensemble :

$$H(S) = - \sum p_i \log_2(p_i)$$

Où p_i est la proportion des éléments de classe i .

- Si $H(S) = 0 \rightarrow$ ensemble pur (une seule classe).
- Si $H(S) = 1 \rightarrow$ mélange parfait de classes.

Arbres de Décision

Mesure de l'impureté – Indice de Gini

- L'indice de Gini mesure la probabilité d'une classification incorrecte :

$$Gini(S) = 1 - \sum p_i^2$$

- Si Gini = 0 → ensemble pur.
 - Si Gini = 0.5 → forte diversité.
- **Comparaison Entropie vs Gini :**
 - **Entropie** plus précise mais plus coûteuse en calcul.
 - **Gini** plus rapide mais légèrement moins précis.

Random Forest

Random Forest

- Random Forest est un algorithme d'apprentissage supervisé utilisé pour la classification et la régression.
- Il est basé sur un ensemble d'arbres de décision entraînés sur différentes sous-parties des données.
- L'objectif est d'améliorer la précision et de réduire le surapprentissage par rapport aux arbres de décision individuels.
- **Principe des Arbres de Décision**
 - Un arbre de décision divise l'espace des caractéristiques en sous-espaces **hiérarchiquement organisés**.
 - Chaque nœud correspond à un **test** sur une variable et les feuilles contiennent les **prédictions finales**.
 - La séparation est basée sur des **critères d'impureté** comme :
 - **Gini** : $G = 1 - \sum p_i^2$
 - **Entropie** : $H = -\sum p_i \log_2 p_i$

Random Forest

Concepts Clés du Random Forest

1. Apprentissage Ensembliste

1. Combine les prédictions de plusieurs modèles (arbres de décision) pour améliorer la généralisation.
2. Utilise le **bagging (Bootstrap Aggregating)** pour réduire la variance.

2. Arbres de Décision

1. Chaque arbre divise les données selon des seuils de caractéristiques (ex : indice Gini ou entropie pour la classification, MSE pour la régression).
2. Les arbres sont **profonds** (faible biais) mais moyennés pour réduire la variance.

3. Sélection Aléatoire des Caractéristiques

1. Chaque arbre utilise un **sous-ensemble aléatoire de caractéristiques** ($\sqrt{n}$ caractéristiques pour la classification, $n/3$ pour la régression).
2. Introduit de la **diversité** entre les arbres, améliorant la robustesse.

Random Forest

Fonctionnement du Random Forest

1. Phase d'Entraînement

- **Étape 1** : Crée des **jeux de données bootstrap** (échantillonnage avec remise).
- **Étape 2** : Entraîne plusieurs arbres sur différents sous-ensembles de données et caractéristiques.
- **Étape 3** : Chaque arbre fait des prédictions indépendantes.

2. Phase de Prédiction

- **Classification** : Vote majoritaire de tous les arbres.
- **Régression** : Moyenne de toutes les prédictions.

Random Forest

Fondements Mathématiques

- Indice de Gini (Classification)

$$Gini = 1 - \sum_{i=1}^C (p_i)^2$$

- où p_i = probabilité de la classe i .

- Entropie (Alternative)

$$Entropie = - \sum_{i=1}^C p_i \log_2(p_i)$$

- Erreur Quadratique Moyenne (Régression)

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Random Forest

Avantages & Inconvénients

1. Avantages du Random Forest

- Réduit le surajustement (vs. un seul arbre de décision).
- Gère bien les données multidimensionnelles (sélection implicite des caractéristiques).
- Fonctionne avec des valeurs manquantes et des outliers.
- Parallélisable (les arbres sont indépendants).

1. Inconvénients

- Moins interprétable qu'un seul arbre de décision.
- Temps de prédiction plus lent que les modèles linéaires (mais entraînement plus rapide que le boosting comme XGBoost).
- Peut surajuster sur des données bruitées si les arbres sont trop profonds.

Méthodes d'ensemble

Méthodes d'ensemble

Définition :

- En Machine Learning, il existe plusieurs **méthodes d'ensemble** (« *ensemble methods* ») qui combinent plusieurs modèles pour améliorer les performances. Voici un tour d'horizon clair des plus connus :
 - Bagging (Bootstrap Aggregating)
 - Boosting
 - Stacking (Stacked Generalization)
 - Voting (Voting Classifier)

Bagging (Bootstrap Aggregating)

Définition :

- Le **Bagging** consiste à entraîner **plusieurs modèles** (généralement le même type, comme des arbres de décision) sur **différents sous-échantillons** du dataset d'entraînement, puis à **moyenner ou voter** sur leurs résultats.
 - Chaque sous-échantillon est obtenu par **bootstrap** : un échantillonnage **aléatoire avec remise**.
 - Les modèles sont **entraînés en parallèle**.
 - La sortie finale est :
 - Moyenne des prédictions (régression)
 - Vote majoritaire (classification)

Exemple :

- Le **Random Forest** est un exemple typique de bagging appliqué aux arbres de décision.

Boosting

Définition :

- Le **Boosting** construit des modèles **séquentiellement**, chaque nouveau modèle essaie de **corriger les erreurs** du modèle précédent.
 - On donne plus de poids aux erreurs précédentes.
 - Les modèles sont **faibles** (comme de petits arbres) mais deviennent puissants ensembles.
 - Apprentissage **séquentiel** (dépendance entre les modèles).
- **Algorithmes populaires :**
 - AdaBoost
 - Gradient Boosting Machines (GBM)
 - XGBoost
 - LightGBM
 - CatBoost

Exemple :

- Prédiction de crédit bancaire avec **XGBoost**.

Stacking (Stacked Generalization)

Définition :

- Le **Stacking** combine **différents types de modèles** (SVM, KNN, arbres, etc.). Les sorties de ces modèles deviennent les **entrées d'un modèle final** (métamodèle) qui prend la décision.
 - Contrairement au bagging et boosting, les modèles de base peuvent être **hétérogènes**.
 - Apprentissage **en 2 niveaux** : base + métamodèle.

Exemple :

- Utiliser une combinaison de modèles (SVM, RF, XGBoost) avec une régression logistique comme métamodèle final.

Voting (Voting Classifier)

Définition :

- Méthode simple où plusieurs modèles (différents ou identiques) votent pour la classe majoritaire (classification) ou la moyenne (régression).
 - **Voting dur (hard voting)** : vote majoritaire
 - **Voting doux (soft voting)** : moyenne pondérée des probabilités

Exemple :

- Combiner les prédictions de KNN, SVM et Random Forest par vote majoritaire.

Tableau Comparatif des Méthodes d'Ensemble

Méthode	Entraînement	Type de modèles	Idée principale
Bagging	Parallèle	Identiques	Réduire la variance
Boosting	Séquentiel	Identiques (faibles)	Réduire biais + variance
Stacking	Parallèle + 1 méta	Différents	Apprendre à combiner plusieurs algorithmes
Voting	Parallèle	Différents/Identiques	Vote ou moyenne des prédictions

Autres Algorithmes

Apprentissage Supervisé	Apprentissage Non-Supervisé	Apprentissage par Renforcement	Réduction de Dimension	Algorithmes d'optimisation
Gradient Boosting Machines (GBM)	K-Means Clustering	Q-Learning	Principal Component Analysis (PCA)	Genetic Algorithms
XGBoost	DBSCAN	Deep Q-Network (DQN)	t-SNE	Ant Colony Optimization
LightGBM	Mean Shift	SARSA	UMAP	Artificial Bee Colony
CatBoost	Gaussian Mixture Models (GMM)	Policy Gradient Methods		Particle Swarm Optimization
	Agglomerative Clustering (Hiérarchique)			Grey Wolf Optimizer
	Self-Organizing Maps (SOM)			Whale Optimization Algorithm
	Naive Bayes			